

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG

-----***-----



UIT
TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

**ĐỀ TÀI: HỆ THỐNG GIÁM SÁT NHIỆT ĐỘ, ĐỘ ẨM
VÀ CẢNH BÁO TỪ XA SỬ DỤNG ESP8266**

Giảng viên hướng dẫn: **ThS. Lê Anh Tuấn**

Lớp: **NT131.P22**

Nhóm thực hiện: **Nhóm 14**

23520541 – Trần Nguyễn Việt Hoàng

23521572 – Lê Quang Tiến

23521778 – Lê Hoàng Việt

MỤC LỤC

Chương 1. TỔNG QUAN	1
1.1. Mục tiêu	1
1.2. Phần cứng	1
1.2.1. Module ESP8266	1
1.2.2. Cảm biến DHT11	3
1.2.3. Cảm biến Relay 5V	5
1.3. Phần mềm	6
1.3.1. Ardunio IDE	6
1.3.2. ThingSpeak.....	8
1.3.3. Supabase	10
Chương 2. THIẾT KẾ HỆ THỐNG	11
2.1. Lắp ráp sơ đồ mạch	11
2.2. Viết chương trình	14
2.2.1. Cài đặt chức năng cho ESP8266 và gửi dữ liệu lên ThingSpeak.....	14
2.2.2. Xây dựng cơ sở dữ liệu trên Supabase.....	23
2.2.3. Sử dụng .NET Framework để xây dựng giao diện người dùng	25
Chương 3. TRIỂN KHAI VÀ DEMO.....	58
3.1. Hình ảnh minh họa	58
3.2. Video demo.....	66
Chương 4. KẾT LUẬN.....	66
4.1. Nhận xét về đồ án.....	66
4.2. Hướng phát triển sau này.....	67

4.3.	Đường dẫn chứa các file trong báo cáo.....	67
------	--	----

LỜI MỞ ĐẦU

Trong bối cảnh khoa học công nghệ ngày càng phát triển, con người ngày càng sáng tạo ra nhiều vật dụng, thiết bị thông minh và hiện đại hơn nhằm đáp ứng các nhu cầu cần thiết của con người. Mỗi giai đoạn phát triển của lịch sử thế giới đều gắn liền với những cuộc cách mạng về khoa học kỹ thuật. Ngày nay, cuộc cách mạng Internet of Things đã tạo nên những thay đổi chóng mặt cho cuộc sống con người ở hiện tại và trong tương lai. IoT được ứng dụng vào rất nhiều mặt của cuộc sống, trong công nghiệp, nông nghiệp, giáo dục và y tế... Việc giám sát sự thay đổi của môi trường, và việc thu thập và đánh giá nhiệt độ độ ẩm môi trường cũng là một phần trong đó. Xây dựng các hệ thống tự động hóa để theo dõi các thông số như nhiệt độ và độ ẩm trở nên thiết yếu trong nhiều lĩnh vực như nông nghiệp, sản xuất, và bảo quản. Đồ án "Hệ thống giám sát nhiệt độ, độ ẩm và cảnh báo từ xa sử dụng ESP8266" được thực hiện nhằm thiết kế và phát triển một giải pháp hiệu quả, chi phí thấp để đo lường, hiển thị, điều khiển và cảnh báo các thay đổi của nhiệt độ và độ ẩm một cách thuận lợi và hiệu quả. Đồ án không chỉ tập trung vào việc triển khai phần cứng và phần mềm mà còn hướng tới việc cung cấp một nền tảng mở, dễ mở rộng, phù hợp cho các ứng dụng thực tế. Thông qua quá trình thực hiện, người thực hiện mong muốn nâng cao kỹ năng lập trình nhúng, hiểu biết về IoT, và áp dụng kiến thức vào giải quyết các vấn đề thực tiễn trong cuộc sống.

Chương 1. TỔNG QUAN

1.1. Mục tiêu

Sử dụng Module ESP8266 để xây dựng một hệ thống cho phép thu thập và theo dõi thông số môi trường (nhiệt độ và độ ẩm) từ bất kì đâu thông qua mạng Internet, giúp người dùng kiểm soát tình trạng môi trường mà không cần có mặt trực tiếp.

Hiển thị dữ liệu thu thập được từ môi trường (nhiệt độ và độ ẩm) và thống kê dữ liệu qua từng thời điểm. Đồng thời, tích hợp tính năng cảnh báo từ xa (qua âm thanh hoặc thông báo đầy) khi nhiệt độ hoặc độ ẩm vượt ngưỡng cho phép. Và sử dụng công cụ AI để đưa ra đề xuất và tự động hóa mọi việc.

1.2. Phần cứng

1.2.1. Module ESP8266

Khái niệm

ESP8266 là một module Wi-Fi với khả năng kết nối Internet và được tích hợp sẵn trên một số board nhúng như NodeMCU, Wemos, và ESP-01. ESP8266 có thể hoạt động như một điểm truy cập (access point), có khả năng giao tiếp với nhiều loại thiết bị khác nhau thông qua mạng không dây. [1]

ESP 8266 được sử dụng rộng rãi trong các ứng dụng IoT (Internet of Things) như cảm biến thông minh, hệ thống kiểm soát thiết bị, hoặc các ứng dụng điều khiển từ xa. Module này có giá thành rẻ và rất dễ sử dụng, cùng với đó là khả năng tương thích với nhiều loại vi điều khiển khác nhau.

ESP8266 WIFI NodeMCU

ESP8266 NodeMCU là một nền tảng IoT mã nguồn mở. NodeMCU cung cấp một bộ SDK để lập trình cho ESP8266 bằng ngôn ngữ C++. Một số tính năng như Wi-Fi, GPIO, ADC, I2C, SPI, PWM và các tính năng khác. [1]

Sơ đồ chân ESP8266 NodeMCU

Các tính năng nổi bật

- Khả năng kết nối WiFi: ESP8266 hỗ trợ chuẩn WiFi 802.11 b/g/n, cho phép kết nối Internet dễ dàng mà không cần module WiFi rời.
- Vi điều khiển tích hợp: Với CPU Tensilica L106 32-bit, tốc độ lên đến 80 MHz (có thể ép xung lên 160 MHz), ESP8266 không chỉ đóng vai trò như một module giao tiếp mà còn có thể hoạt động độc lập như một vi điều khiển.
- GPIO đa dụng: Module cung cấp nhiều chân GPIO (General Purpose Input/Output) để kết nối với cảm biến, thiết bị ngoại vi như LED, buzzer, hoặc màn hình.
- Dung lượng bộ nhớ: Có 1MB Flash (phiên bản cơ bản) để lưu trữ chương trình, đủ cho các ứng dụng IoT đơn giản.
- Chi phí thấp: Giá thành rẻ (thường dưới 5 USD cho các phiên bản như NodeMCU), phù hợp với các dự án học tập và thử nghiệm.
- Hỗ trợ cộng đồng: ESP8266 được hỗ trợ rộng rãi bởi Arduino IDE và nhiều thư viện mã nguồn mở, giúp người dùng dễ dàng lập trình.

Vai trò trong dự án

Trong dự án này, ESP8266 (dạng NodeMCU) được sử dụng làm trung tâm điều khiển, thu thập dữ liệu từ cảm biến DHT11/DHT22, kết nối với mạng WiFi, và gửi thông tin đến nền tảng Blynk để giám sát từ xa.

1.2.2. Cảm biến DHT11

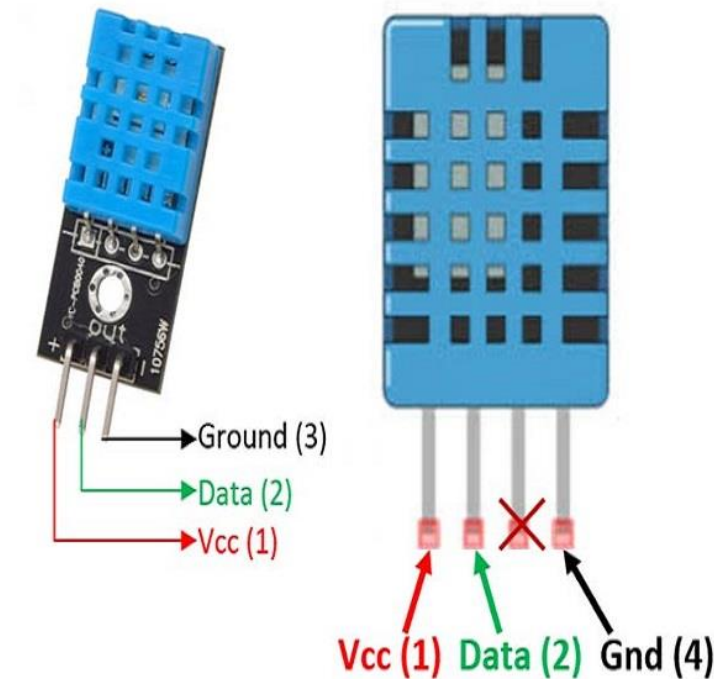
Khái niệm

DHT11 là một cảm biến kỹ thuật số dùng để cảm nhận nhiệt độ và độ ẩm. Có thể dễ dàng giao tiếp với bất kỳ điều khiển vi nào như Arduino, Raspberry Pi, ... để đo độ ẩm và nhiệt độ ngay lập tức. [2]

Để đo không khí xung quanh, cảm biến DHT11 này sử dụng một điện trở nhiệt và một cảm biến độ ẩm điện dung.

DHT11 lấy dữ liệu thông qua giao tiếp 1 wire (giao tiếp digital 1 dây truyền dữ liệu duy nhất). Bộ tiền xử lý tín hiệu tích hợp trong cảm biến giúp ta có được dữ liệu chính xác mà không phải qua bất kỳ tính toán nào. [2]

Sơ đồ chân DHT11



Số chân	Tên chân	Mô tả
1	Vcc	Nguồn 3.5V đến 5.5V
2	Data	Đầu ra cả nhiệt độ và độ ẩm thông qua dữ liệu nối tiếp
3	NC	Không có kết nối và do đó không sử dụng
4	Ground	Nối đất

Thông số kỹ thuật DHT11

- Điện áp hoạt động: 3V - 5V DC
- Dòng điện tiêu thụ: 2.5mA
- Phạm vi cảm biến độ ẩm: 20% - 90% RH, sai số $\pm 5\%RH$

- Phạm vi cảm biến nhiệt độ: $0^{\circ}\text{C} \sim 50^{\circ}\text{C}$, sai số $\pm 2^{\circ}\text{C}$
- Tần số lấy mẫu tối đa: 1Hz (1 giây 1 lần)
- Kích thước: $23 * 12 * 5 \text{ mm}$

Vai trò trong đồ án

Trong đồ án này, DHT11 đóng vai trò làm một cảm biến đo nhiệt độ và độ ẩm để cho ESP8266 có thể thu thập dữ liệu, so sánh và gửi cảnh báo nếu như dữ liệu đo được vượt mức quá cho phép (30 độ).

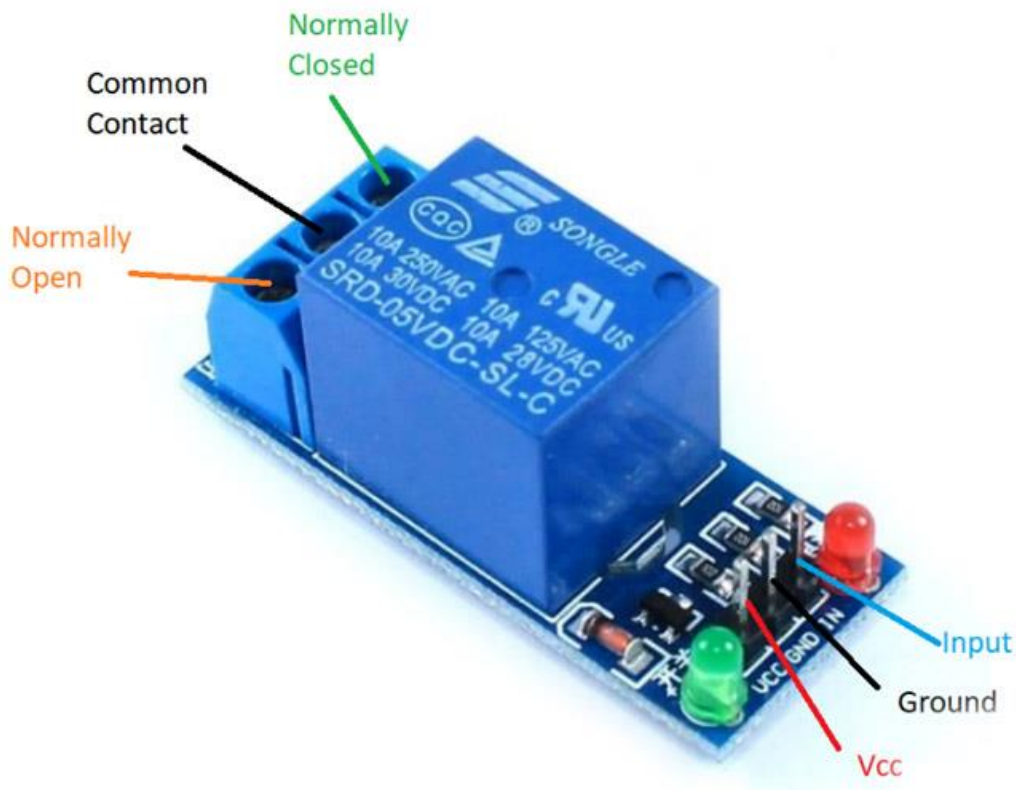
1.2.3. Cảm biến Relay 5V

Khái niệm

Module cảm biến ánh sáng Relay 5V là một công tắc ba chiều (COM, NO và NC) sử dụng cường độ ánh sáng xung quanh để điều khiển trực tiếp tải đầu ra. Chiết áp điều chỉnh của nó cho phép ta thay đổi độ nhạy và đặt ngưỡng khởi động role. [3]

Sơ đồ chân

- Vcc+: Cực dương của nguồn điện 5V DC
- GND-: Cực âm của nguồn điện 0V DC
- Cầm cảm biến : Không phân biệt cực
- Biến trở : Chỉnh độ nhạy của cảm biến
- Đầu cuối thường đóng (NC): Đầu cuối thường đóng của role
- Đầu cuối chung (COM): Đầu cuối chung cực của role
- Cực mở thường (NO): Cực mở thường xuyên của role



Thông số kỹ thuật

- IC sử dụng : LM393
- Điện áp hoạt động: DC5V/12V
- Có led báo nguồn và led báo tín hiệu
- Ngõ ra: tiếp điểm Relay
- Độ nhạy có thể điều chỉnh được bằng chiết áp
- Kích thước PCB: 49 * 26 * 17mm

Vai trò trong đồ án

Trong đồ án lần này, Relay đóng vai trò như một công tắc dùng để bật/tắt các thiết bị (Ở đồ án này là quạt 4010 5V) dựa trên điều kiện nhiệt độ cũng như độ ẩm đo được từ cảm biến DHT11

1.3. Phần mềm

1.3.1. Arduino IDE



Arduino IDE 2.0.4

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux AppImage 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.14: "Mojave" or newer, 64 bits

macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Khái niệm

Arduino là nền tảng mã nguồn mở giúp con người xây dựng các ứng dụng điện tử có khả năng liên kết, tương tác với nhau tốt hơn. Arduino có thể xem như một chiếc máy tính thu nhỏ giúp người dùng lập trình, thực hiện các dự án điện tử không cần tới công cụ chuyên biệt phục cho quá trình nạp code. [4]

Arduino IDE là môi trường lập trình hỗ trợ ngôn ngữ C/C++ đơn giản hóa (dựa trên Wiring) được sử dụng để viết và biên dịch mã vào module Arduino.

Đặc điểm

Arduino IDE sử dụng ngôn ngữ lập trình C/C++ rất phổ biến trong giới lập trình. Bất kỳ đoạn code nào của C/C++ thì Arduino IDE đều có thể nhận dạng, giúp các lập trình viên thuận tiện trong việc thiết kế chương trình lập cho các bo mạch Arduino.

Arduino có một module quản lý bo mạch, nơi người dùng có thể chọn bo mạch mà họ muốn làm việc cùng và có thể thay đổi bo mạch thông qua Menu. Quá trình sửa đổi lựa chọn cũng liên tục tự động cập nhật để các dữ liệu có sẵn trong bo mạch và dữ liệu sửa đổi đồng nhất với nhau. [4]

Arduino IDE hoạt động trên 3 hệ điều hành phổ biến nhất là Windows, Mac OS và Linux giúp người dùng có thể truy cập vào phần mềm ở bất cứ đâu, bất cứ khi nào.

Vai trò trong dự án

Trong dự án này, Arduino IDE được nhóm sử dụng để lập trình ESP8266. Các thư viện như DHT, ESP8266WiFi được tích hợp vào Arduino IDE, giúp module thu thập và xử lý dữ liệu, đơn giản hóa quá trình đọc dữ liệu từ cảm biến, kết nối WiFi và giao tiếp với nền tảng IoT thông qua thư viện ThingSpeak.

1.3.2. ThingSpeak

Khái niệm

ThingSpeak là một nền tảng IoT (Internet of Things) dựa trên đám mây cho phép người dùng thu thập, lưu trữ, phân tích và trực quan hóa dữ liệu được tạo ra từ các thiết bị hoặc cảm biến kết nối mạng. [5]

Với ThingSpeak, người dùng có thể dễ dàng xác định cấu hình các thiết bị để gửi dữ liệu đến nền tảng thông qua các giao thức truyền thông mạng như HTTP, MQTT và nhiều giao thức khác. Một khi dữ liệu được tải lên, nó có thể được xử lý, phân tích và trực quan hóa thông qua các biểu đồ và biểu mẫu trực tuyến, cho phép người dùng theo dõi và đánh giá xu hướng dữ liệu trong thời gian thực. [5]

Tính năng

Thu thập và lưu trữ dữ liệu:

- Gửi dữ liệu từ các thiết bị IoT lên đám mây bằng nhiều giao thức khác nhau.
- Lưu trữ dữ liệu trong các kênh riêng tư hoặc công khai.
- Hỗ trợ nhiều loại dữ liệu, bao gồm số, chuỗi, ngày giờ và vị trí.

Phân tích dữ liệu:

- Cung cấp các công cụ tích hợp để phân tích dữ liệu, bao gồm các biểu đồ, bảng điều khiển và widget.
- Cho phép sử dụng MATLAB® để thực hiện phân tích nâng cao hơn.

- Hỗ trợ nhiều loại phân tích, bao gồm thống kê, học máy và xử lý tín hiệu.

Trực quan hóa dữ liệu:

- Cung cấp các công cụ trực quan hóa mạnh mẽ để giúp bạn hiểu rõ hơn về dữ liệu của mình.
- Tạo biểu đồ, bảng điều khiển và widget để hiển thị dữ liệu của bạn theo nhiều cách khác nhau.
- Hỗ trợ nhiều loại trực quan hóa, bao gồm biểu đồ đường, biểu đồ cột, biểu đồ pie và bản đồ.

Phát triển ứng dụng IoT:

- Cung cấp một API RESTful cho phép bạn xây dựng các ứng dụng IoT tùy chỉnh.
- Sử dụng API để truy cập dữ liệu, điều khiển thiết bị và xây dựng các bảng điều khiển tùy chỉnh.
- Hỗ trợ nhiều ngôn ngữ lập trình, bao gồm Python, Java, JavaScript và C++.

Cách hoạt động

- Gửi dữ liệu: Dữ liệu từ các thiết bị IoT của bạn được gửi đến ThingSpeak thông qua API RESTful.
- Lưu trữ dữ liệu: Dữ liệu được lưu trữ trong kênh ThingSpeak của bạn.
- Xử lý dữ liệu: Bộ xử lý ThingSpeak của bạn được chạy để phân tích và xử lý dữ liệu.
- Hiển thị dữ liệu: Dữ liệu có thể được hiển thị trên các biểu đồ, bảng điều khiển và widget.
- Điều khiển thiết bị: Dữ liệu có thể được sử dụng để điều khiển các thiết bị IoT khác.

Vai trò trong dự án

Thingspeak được sử dụng để lấy dữ liệu nhận được từ module ESP8266, sau đó trích xuất dữ liệu dưới dạng JSON và có thể được gọi thông qua API. Nhóm sẽ tự thiết kế một phần

mềm quản lý thiết bị người dùng và sử dụng dữ liệu nhận từ ThingSpeak để hiển thị các thông số nhiệt độ, độ ẩm và các trường thông tin khác lên giao diện.

1.3.3. Supabase

Khái niệm

Supabase là một nền tảng mã nguồn mở cung cấp backend-as-a-service (BaaS), giúp cho lập trình viên có thể dễ dàng triển khai các ứng dụng có backend mạnh mẽ. Nó bao gồm cơ sở dữ liệu PostgreSQL, xác thực người dùng, API thời gian thực và lưu trữ file. [5]

Đặc điểm

Mỗi dự án Supabase đi kèm với một cơ sở dữ liệu PostgreSQL đầy đủ, cho phép truy vấn SQL linh hoạt.

Tích hợp sẵn xác thực người dùng (email/password, OAuth, v.v.) với hỗ trợ Row Level Security (RLS) để bảo mật dữ liệu.

Cung cấp API RESTful tự động cho mọi bảng trong cơ sở dữ liệu, cùng với tài liệu API được tạo tự động.

Hỗ trợ lưu trữ tệp (tương thích S3), đồng bộ hóa dữ liệu thời gian thực, và các hàm Edge Functions để xử lý logic backend.

Là mã nguồn mở, cho phép tự host hoặc tùy chỉnh theo nhu cầu, tránh phụ thuộc vào nhà cung cấp (vendor lock-in).

Chức năng

Cơ sở dữ liệu PostgreSQL:

- Mỗi dự án có một cơ sở dữ liệu PostgreSQL được quản lý hoàn toàn, cho phép lưu trữ dữ liệu có cấu trúc với mối quan hệ phức tạp.
- Người dùng có thể quản lý dữ liệu qua giao diện dashboard, SQL Editor, hoặc API, với các extension (như pgvector cho AI) dễ kích hoạt.

Xác thực (Authentication):

- Sử dụng JSON Web Tokens (JWT) để xác thực người dùng và tích hợp với RLS để kiểm soát truy cập hàng loạt.
- Hỗ trợ nhiều phương thức đăng nhập (email, mạng xã hội) và lưu trữ thông tin người dùng trong một schema đặc biệt trong PostgreSQL.

API tự động:

- Dựa trên PostgREST, Supabase tạo API RESTful cho mọi bảng mà không cần viết mã thủ công.
- API tự động cập nhật khi cơ sở dữ liệu thay đổi

Thời gian thực (Realtime):

- Sử dụng WebSocket để phát trực tiếp các thay đổi trong cơ sở dữ liệu (thêm, sửa, xóa) đến ứng dụng, phù hợp cho chat, thông báo, hoặc bảng trắng chia sẻ.

Lưu trữ (Storage):

- Tích hợp với dịch vụ lưu trữ tương thích S3, cho phép lưu và phục vụ tệp lớn (hình ảnh, video) với chính sách bảo mật RLS.

Quản lý và triển khai:

- Cung cấp dashboard trực quan để quản lý dự án, bảng dữ liệu, và người dùng.
- Hỗ trợ tự host với Docker hoặc sử dụng dịch vụ đám mây của Supabase

Vai trò đồ án

Trong đồ án, Supabase đóng vai trò như một cơ sở dữ liệu PostgreSQL được quản lý, lưu trữ thông tin người dùng và thiết bị, hỗ trợ các chức năng đăng nhập, khôi phục mật khẩu, và quản lý dữ liệu động.

Chương 2. THIẾT KẾ HỆ THỐNG

2.1. Lắp ráp sơ đồ mạch

Thành phần

ESP8266: Là vi điều khiển chính, cung cấp khả năng xử lý, kết nối Wi-Fi và giao tiếp với các thiết bị khác.

Cảm biến DHT11: Đo nhiệt độ và độ ẩm môi trường.

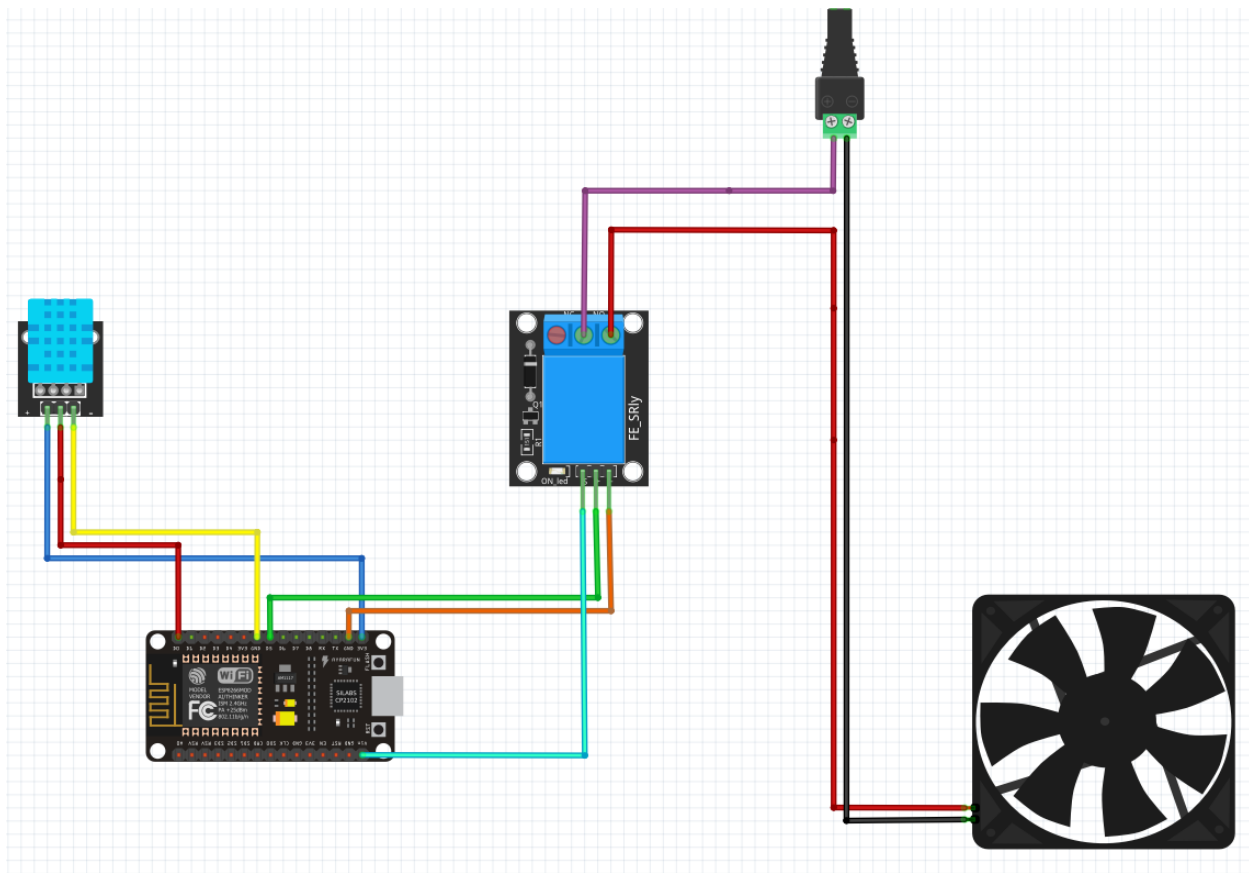
Rơ-le 5VDC: Dùng để điều khiển quạt bằng cách đóng/mở mạch nguồn.

Quạt (Fan): Thiết bị được điều khiển để làm mát khi nhiệt độ vượt ngưỡng.

Nguồn điện (Barrel Jack): Cung cấp điện cho quạt thông qua rơ-le.

Thông số kỹ thuật

- ESP8266 → DHT11
- 3V → VCC
- D0 → OUT
- G → GND
- ESP8266 → Relay 5VDC
- VIN → VCC
- D5 → IN
- G → GND
- Relay 5VDC → Fan
- ON → 5V
- Relay5VDC → Barrel Jack
- COM → POS
- Fan → Barrel Jack
- GND → NEG



Mô tả kết nối giữa các thành phần

Kết nối giữa ESP8266 và DHT11:

- ESP8266 3V → DHT11 VCC: Nguồn điện 3.3V từ ESP8266 cung cấp cho cảm biến DHT11.
- ESP8266 D0 (GPIO 16) → DHT11 OUT: Chân dữ liệu (OUT) của DHT11 được kết nối với chân D0 của ESP8266 để đọc dữ liệu nhiệt độ và độ ẩm.
- ESP8266 G (GND) → DHT11 GND: Chân GND của ESP8266 nối với chân GND của DHT11 để hoàn thiện mạch.

Kết nối giữa ESP8266 và Rơ-le 5VDC:

- ESP8266 VIN → Rơ-le 5VDC VCC: Nguồn điện 5V từ ESP8266 (thường được cấp qua cổng VIN hoặc nguồn bên ngoài) cung cấp cho rơ-le.

- ESP8266 D5 (GPIO 14) → Rơ-le 5VDC IN: Chân điều khiển (IN) của rơ-le được kết nối với chân D5 của ESP8266 để điều khiển rơ-le bật/tắt.
- ESP8266 G (GND) → Rơ-le 5VDC GND: Chân GND của ESP8266 nối với chân GND của rơ-le

Kết nối giữa Rơ-le 5VDC và Quạt:

- Rơ-le 5VDC ON → Barrel Jack 5V: Chân ON (hoặc NO - Normally Open) của rơ-le nối với cực dương (5V) của nguồn Barrel Jack.
- Rơ-le 5VDC COM → Barrel Jack POS: Chân COM (Common) của rơ-le nối với cực dương (POS) của nguồn Barrel Jack để cấp điện cho quạt khi rơ-le đóng.
- Quạt GND → Barrel Jack NEG: Chân GND của quạt nối với cực âm (NEG) của nguồn Barrel Jack để hoàn thiện mạch nguồn của quạt.

Kết nối giữa Rơ-le 5VDC và nguồn Barrel Jack: COM của relay 5VDC được nối với cực dương (POS) của Barrel Jack để điều khiển dòng điện từ nguồn DC. Mục đích kết nối là dùng relay làm công tắc để bật/tắt nguồn DC cấp cho thiết bị.

Nguồn Barrel Jack cung cấp điện cho quạt thông qua rơ-le. ESP8266 có thể được cấp nguồn từ USB hoặc nguồn bên ngoài (thông qua VIN)

2.2. Viết chương trình

2.2.1. Cài đặt chức năng cho ESP8266 và gửi dữ liệu lên ThingSpeak

Thư viện sử dụng

- *<ESP8266WiFi.h>*: cho phép ESP8266 kết nối với mạng Wi-Fi.
- *<ThingSpeak.h>*: cung cấp giao diện để gửi dữ liệu lên ThingSpeak để lưu trữ và phân tích dữ liệu.
- *"DHTesp.h"*: thư viện để giao tiếp với cảm biến DHT11/DHT22 để đo nhiệt độ và độ ẩm.
- *<Wire.h>*: thư viện giao tiếp I2C, cần thiết cho màn hình OLED.

- `<Adafruit_GFX.h>` và `<Adafruit_SSD1306.h>`: thư viện để điều khiển màn hình OLED SSD1306.

```
1  #include <ESP8266WiFi.h>
2  #include <ThingSpeak.h>
3  #include "DHTesp.h"
4  #include <Wire.h>
5  #include <Adafruit_GFX.h>
6  #include <Adafruit_SSD1306.h>
```

Định nghĩa các giá trị hằng số

Wi-Fi và ThingSpeak:

- *ssid* và *password*: tên và mật khẩu của mạng Wi-Fi ("MANGCUATHANG" và "EOCHOXAI").
- *myChannelNumber* = 2943515: ID kênh ThingSpeak để gửi dữ liệu.
- *myWriteAPIKey* = "3UG68C385KKESWVG": khóa API để ghi dữ liệu lên ThingSpeak.

OLED:

- *SCREEN_WIDTH* = 128 và *SCREEN_HEIGHT* = 32: Kích thước màn hình OLED (pixel).
- *OLED_RESET* = -1: Không sử dụng chân reset riêng.
- *SCREEN_ADDRESS* = 0x3C: Địa chỉ I2C của màn hình OLED.

DHT và Relay:

- *temperature* và *humidity*: Lưu giá trị nhiệt độ và độ ẩm từ DHT11.
- *tempThreshold* = 30.0: Ngưỡng nhiệt độ (30°C) để bật quạt.
- *fanPin* = D5: Replay nối chân D5 (GPIO14)

Timer:

- *sendInterval* = 20000: Khoảng thời gian gửi dữ liệu lên ThingSpeak (20 giây).

- *lastDhtReadTime* và *lastSendTime*: Theo dõi thời gian đọc dữ liệu từ DHT11 và thời gian gửi lên ThingSpeak.

```

8 // WiFi & ThingSpeak
9 const char* ssid = "MANGCUATHANG";
10 const char* password = "EOCHOXAI";
11 WiFiClient client;
12
13 unsigned long myChannelNumber = 2943515;
14 const char* myWriteAPIKey = "3UG68C385KKESWVG";
15
16 // OLED
17 #define SCREEN_WIDTH 128
18 #define SCREEN_HEIGHT 32
19 #define OLED_RESET -1
20 #define SCREEN_ADDRESS 0x3C
21 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
22
23 // DHT
24 DHTesp dht;
25 float temperature, humidity;
26 unsigned long lastDhtReadTime = 0;
27
28 // Relay/Quạt
29 const int fanPin = D5;
30 const float tempThreshold = 30.0;
31
32 // Timer
33 unsigned long lastSendTime = 0;
34 const long sendInterval = 20000;

```

Hàm setup(): được thực thi mỗi lần khi ESP8266 khởi động

Serial.begin(115200): bật cổng Serial với tốc độ 115200 baud để debug.

Khởi tạo Relay:

- *pinMode(fanPin, OUTPUT)*: Định nghĩa chân D5 là đầu ra.
- *digitalWrite(fanPin, LOW)*: tắt quạt ban đầu (rơ-le ở trạng thái LOW để ngắt mạch).

Khởi động kết nối Wi-Fi:

- *WiFi.begin(ssid, password)*: kết nối đến mạng Wi-Fi với giá trị ssid và password trước đó

- Vòng lặp *while (WiFi.status() != WL_CONNECTED)*: chờ đến khi kết nối thành công, in dấu chấm (.) mỗi 500ms để báo hiệu.

ThingSpeak.begin(client): khởi tạo kết nối với ThingSpeak.

if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)): khởi tạo màn hình OLED SSD1306 sử dụng giao thức I2C.

- Xử lý lỗi bằng cách in ra thông báo “Không khởi tạo được OLED” và dừng chương trình bằng *while (true)* để đảm bảo rằng màn hình OLED không hoạt động
- *display.display()*: gửi dữ liệu từ bộ đệm (buffer) của thư viện Adafruit_SSD1306 đến màn hình OLED và chờ 2s *delay(2000)* để hiển thị

dht.setup(16, DHTesp::DHT11): cấu hình DHT11 kết nối với chân D0 (GPIO 16).

```

36 void setup() {
37     Serial.begin(115200);
38     delay(100);
39
40     // Khởi động relay
41     pinMode(fanPin, OUTPUT);
42     digitalWrite(fanPin, LOW); // Tắt quạt ban đầu
43
44     // Khởi động WiFi
45     WiFi.begin(ssid, password);
46     Serial.print("Đang kết nối WiFi");
47     while (WiFi.status() != WL_CONNECTED) {
48         delay(500);
49         Serial.print(".");
50     }
51     Serial.println("\nWiFi đã kết nối");
52
53     // ThingSpeak
54     ThingSpeak.begin(client);
55
56     // OLED
57     if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
58         Serial.println(F("Không khởi tạo được OLED"));
59         while (true);
60     }
61     display.display();
62     delay(2000);
63
64     // DHT11
65     dht.setup(16, DHTesp::DHT11);
66 }
67

```

Hàm controlFan(): điều khiển quạt trên nhiệt độ

Nếu giá trị nhiệt độ hiện tại $temp >$ giá trị nhiệt độ định mức $tempThreshold$ (30°C), đặt `digitalWrite(fanPin, HIGH)` để bật quạt và in thông báo "Quạt ĐÃ BẬT".

Ngược lại, đặt `digitalWrite(fanPin, LOW)` để tắt quạt và in thông báo "Quạt TẮT".

```

68 void controlFan(float temp) {
69     if (temp > tempThreshold) {
70         digitalWrite(fanPin, HIGH); // Bật quạt (HIGH)
71         Serial.println(">> Quạt ĐÃ BẬT (nhiệt độ cao)");
72     } else {
73         digitalWrite(fanPin, LOW); // Tắt quạt
74         Serial.println(">> Quạt TẮT (nhiệt độ bình thường)");
75     }
76 }

```

Hàm loop(): thực thi đọc và gửi dữ liệu liên tục

if (millis() - lastDhtReadTime > dht.getMinimumSamplingPeriod()): kiểm tra xem đã đủ thời gian để đọc dữ liệu tiếp theo chưa

Đọc dữ liệu từ DHT11:

- Kiểm tra *millis() - lastDhtReadTime > dht.getMinimumSamplingPeriod()* để đảm bảo đọc dữ liệu cách nhau ít nhất thời gian lấy mẫu tối thiểu của DHT11 (khoảng 1-2 giây).
- Lấy giá trị nhiệt độ (t) và độ ẩm (h) từ DHT11.
- Nếu trạng thái OK, cập nhật *temperature* và *humidity*, gọi *showOled()* để hiển thị và *controlFan()* để điều khiển quạt.

lastDhtReadTime = millis(): cập nhật *lastDhtReadTime* bằng thời gian hiện tại để chuẩn bị cho lần đọc tiếp theo

Gửi dữ liệu nhiệt độ và độ ẩm ThingSpeak:

- Kiểm tra *millis() - lastSendTime > sendInterval:* để gửi dữ liệu định kỳ sau 20 giây.
- Gọi *sendToThingSpeak(temperature, humidity):* gửi dữ liệu nhiệt độ và độ ẩm lên các Field trên ThingSpeak
- *lastSendTime = millis():* cập nhật *lastSendTime* bằng thời gian hiện tại để chuẩn bị cho lần gửi tiếp theo

```

77
78 void loop() {
79     if (millis() - lastDhtReadTime > dht.getMinimumSamplingPeriod()) {
80         float t = dht.getTemperature();
81         float h = dht.getHumidity();
82
83         if (dht.getStatusString() == "OK") {
84             temperature = t;
85             humidity = h;
86             showOled(temperature, humidity);
87             controlFan(temperature); // ← Gọi hàm điều khiển quạt tại đây
88         }
89
90         lastDhtReadTime = millis();
91     }
92
93     if (millis() - lastSendTime > sendInterval) {
94         sendToThingSpeak(temperature, humidity);
95         lastSendTime = millis();
96     }
97 }

```

Hàm sendToThingSpeak(): gửi dữ liệu lên ThingSpeak

fanPinState = digitalRead(fanPin): đọc trạng thái hiện tại và lưu giá trị logic của chân D5 (quạt đang chạy HIGH hoặc quạt dừng LOW)

replayState: được sử dụng để biểu thị trạng thái rơ-le dưới dạng 0 hoặc 1

- Nếu *fanPinState == HIGH*, gán *relayState = 1* (quạt bật).
- Nếu *fanPinState == LOW*, gán *relayState = 0* (quạt tắt).

```

99 void sendToThingSpeak(float t, float h) {
100     // Lấy trạng thái hiện tại của chân điều khiển quạt/relay (fanPin)
101     int fanPinState = digitalRead(fanPin); // Đọc trạng thái logic của chân
102
103     // Chuyển đổi trạng thái logic của chân thành 0 (TẮT) hoặc 1 (BẬT)
104     // Dựa trên code gốc của bạn, giả định: HIGH = BẬT quạt (nhiệt độ cao), LOW = TẮT quạt
105     // Nếu chân ở mức CAO (HIGH), trạng thái relay là 1 (BẬT)
106     // Nếu chân ở mức THẤP (LOW), trạng thái relay là 0 (TẮT)
107     int relayState;
108     if (fanPinState == HIGH) {
109         relayState = 1; // Nếu chân ở mức CAO (BẬT), gửi 1 (Biểu thị Relay BẬT)
110     } else {
111         relayState = 0; // Nếu chân ở mức THẤP (TẮT), gửi 0 (Biểu thị Relay TẮT)
112     }

```

In các giá trị sẽ gửi lên Serial Monitor

- `"Sending to ThingSpeak: Temp="`: in chuỗi ký tự để báo hiệu rằng dữ liệu đang được chuẩn bị gửi.
- `Serial.print(t, 2)`: in giá trị nhiệt độ (t) với 2 chữ số thập phân (ví dụ: 25.30).
- `"°C, Hum="`: in đơn vị nhiệt độ (°C) và chuẩn bị in độ ẩm.
- `Serial.print(h, 2)`: in giá trị độ ẩm (h) với 2 chữ số thập phân (ví dụ: 65.50).
- `"%, Relay="`: in đơn vị độ ẩm (%) và chuẩn bị in trạng thái rơ-le.
- `Serial.print(relayState)`: in trạng thái rơ-le (0 hoặc 1).
- `". "`: in dấu chấm và khoảng trắng để kết thúc dòng, giúp thông tin dễ đọc hơn.

```

111 // In ra các giá trị sẽ gửi lên Serial Monitor
112 Serial.print("Sending to ThingSpeak: Temp=");
113 Serial.print(t, 2);
114 Serial.print("°C, Hum=");
115 Serial.print(h, 2);
116 Serial.print("%, Relay=");
117 Serial.print(relayState); // In ra trạng thái 0 hoặc 1
118 Serial.print(". ");
119

```

Gửi dữ liệu lên các Field trên ThingSpeak

- `setField(1, t)`: gửi giá trị nhiệt độ (t) vào Field 1.
- `setField(2, h)`: gửi giá trị độ ẩm (h) vào Field 2.
- `setField(3, relayState)`: gửi trạng thái rơ-le (0 hoặc 1) vào Field 3.

```

121 // Gửi dữ liệu lên các Field trên ThingSpeak
122 ThingSpeak.setField(1, t); // Gửi Nhiệt độ lên Field 1 (Như code gốc)
123 ThingSpeak.setField(2, h); // Gửi Độ ẩm lên Field 2 (Như code gốc)
124 ThingSpeak.setField(3, relayState); // Gửi Trạng thái Relay (0 hoặc 1) lên Field 3
125

```

`ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey)`:

- `myChannelNumber (2943515)`: ID của kênh ThingSpeak mà ta đã tạo.
- `myWriteAPIKey ("3UG68C385KKESWVG")`: khóa API dùng để ghi dữ liệu vào kênh này.
- Hàm này gửi tất cả các Field đã được gán trước đó (Field 1, 2, 3) lên ThingSpeak qua một yêu cầu HTTP POST.

Biến x lưu mã trạng thái HTTP trả về từ ThingSpeak:

- *if* ($x == 200$): nếu mã trạng thái là 200, in thông báo thành công.
- *else*: nếu thất bại, in mã lỗi để người dùng kiểm tra và xử lý.

Serial.println("Đang chờ..."): báo hiệu rằng quá trình gửi dữ liệu đã hoàn tất và chương trình đang chờ đến lần gửi tiếp theo (sau 20 giây, dựa trên *sendInterval*)

```

127   int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
128
129   if (x == 200) {
130       Serial.println("Đã gửi dữ liệu thành công (Mã 200).");
131   } else {
132       Serial.print("Gửi dữ liệu thất bại. Mã lỗi: ");
133       Serial.println(x);
134       // Kiểm tra các mã lỗi phổ biến như đã nêu trước đó.
135   }
136
137   Serial.println("Đang chờ...");
138 }

```

Hàm showOled: hiển thị thông tin nhiệt độ và độ ẩm lên màn hình

- Xóa toàn bộ nội dung trước đó của màn hình
- Đặt kích thước chữ là 2 và chọn màu trắng
- Đặt vị trí con trỏ xuất hiện ban đầu ở góc trên bên trái màn hình
- Hiển thị thông tin nhiệt độ với chữ T để làm kí hiệu, sau đó là giá trị nhiệt độ (lấy 1 số sau dấu phẩy) và đơn vị độ C
- Hiển thị thông tin độ ẩm tương tự như nhiệt độ

```

141 // Hàm hiển thị nhiệt độ và độ ẩm lên OLED
142 void showOled(float t, float h) {
143     display.clearDisplay();
144     display.setTextSize(2);
145     display.setTextColor(SSD1306_WHITE);
146     display.setCursor(0, 0);
147     display.print("T: ");
148     display.print(t, 1);
149     display.print((char)247);
150     display.println("C");
151     display.print("H: ");
152     display.print(h, 0);
153     display.println("%");
154     display.display();
155 }
156

```

2.2.2. Xây dựng cơ sở dữ liệu trên Supabase

Bảng users: dùng để lưu trữ thông tin người dùng trong hệ thống với các thuộc tính

- *id_user SERIAL PRIMARY KEY:* sử dụng id làm khóa chính, SERIAL giúp giá trị của id bắt đầu từ 1 sẽ tự động tăng dần mỗi khi có người dùng được thêm vào.
- *name_user VARCHAR(50) NOT NULL:* lưu tên người dùng, kiểu dữ liệu là chuỗi có độ dài tối đa 50 ký tự, NOT NULL đảm bảo rằng giá trị không được để trống.
- *email VARCHAR(100) UNIQUE NOT NULL:* lưu địa chỉ email của người dùng, UNIQUE đảm bảo rằng không người nào khác có cùng email.
- *phone_number VARCHAR(15) UNIQUE NOT NULL:* lưu số điện thoại của người dùng
- *password VARCHAR(255) NOT NULL:* lưu mật khẩu của người dùng
- *created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP:* lưu thời gian tạo tài khoản, DEFAULT CURRENT_TIMESTAMP sẽ tự động đặt giá trị là lấy thời gian hiện tại khi một người dùng mới được thêm vào

```
1 -- Tạo bảng USERS
2 CREATE TABLE users (
3     id_user SERIAL PRIMARY KEY,
4     name_user VARCHAR(50) NOT NULL,
5     email VARCHAR(100) UNIQUE NOT NULL,
6     phone_number VARCHAR(15) UNIQUE NOT NULL,
7     password VARCHAR(255) NOT NULL,
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
```

Bảng devices: dùng để lưu trữ thông tin thiết bị của người dùng

- *id_device SERIAL PRIMARY KEY:* khóa chính của bảng, tự động tăng mỗi khi thêm thiết bị mới.
- *api_key VARCHAR(255) NOT NULL:* lưu khóa API dùng để xác thực thiết bị và lấy dữ liệu qua Thingspeak.
- *channel_id VARCHAR(255) NOT NULL:* lưu ID kênh dùng để định danh thiết bị.

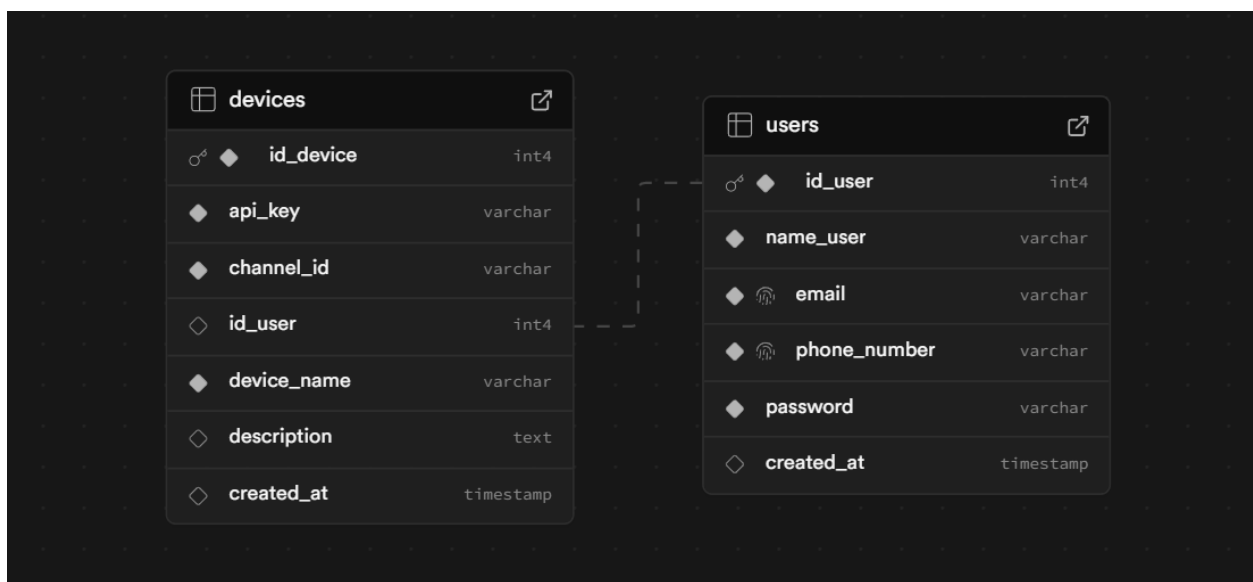
- *id_user INTEGER REFERENCES users(id_user) ON DELETE CASCADE*: lưu ID người dùng sở hữu thiết bị, liên kết với cột id_user trong bảng users.
 - *REFERENCES users(id_user)* tạo mối quan hệ ngoại khóa (foreign key) giữa hai table users và devices
 - *ON DELETE CASCADE* có nghĩa là nếu một người dùng bị xóa khỏi table users, tất cả các thiết bị liên quan trong devices cũng sẽ được xóa tự động.
- *device_name VARCHAR(100) NOT NULL*: lưu tên thiết bị
- *description TEXT*: lưu mô tả chi tiết về thiết bị, kiểu dữ liệu TEXT cho phép nhập văn bản dài tùy ý.
- *created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP*: lưu thời gian tạo thiết bị, tự động đặt là thời gian hiện tại khi thêm bản ghi mới.

```

11 -- Tạo bảng DEVICES
12 CREATE TABLE devices (
13     id_device SERIAL PRIMARY KEY,
14     api_key VARCHAR(255) NOT NULL,
15     channel_id VARCHAR(255) NOT NULL,
16     id_user INTEGER REFERENCES users(id_user) ON DELETE CASCADE,
17     device_name VARCHAR(100) NOT NULL,
18     description TEXT,
19     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
20 );

```

Sơ đồ quan hệ



2.2.3. Sử dụng .NET Framework để xây dựng giao diện người dùng

Form SignUp.cs: cung cấp các phương thức đăng kí cho người dùng

Khai báo biến toàn cục

- *otpStore*: một dictionary lưu trữ mã OTP và thời gian tạo cho từng email, dùng để quản lý và kiểm tra OTP.
- *connString*: chuỗi kết nối đến cơ sở dữ liệu Supabase để kiểm tra email tồn tại.
- *SmtplibUsername* và *SmtplibPassword*: thông tin tài khoản Gmail dùng để gửi email OTP.
- *srcEmail*: lưu email của người dùng đang đăng ký.
- *OTPResendCooldown*: thời gian chờ (60 giây) trước khi gửi lại OTP.
- *OTPExpirationMinutes*: thời gian hết hạn của OTP (5 phút)

```
28     private Dictionary<string, (string otp, DateTime createdTime)> otpStore;  
29     private string connString = "Host=aws-0-ap-southeast-1.pooler.supabase.com;Port=5432;Database=postgres;  
30  
31     private string SmtplibUsername = "sona.feelthemusic@gmail.com";  
32     private string SmtplibPassword = "sxiz nqtc ddko izzj";  
33  
34     private string srcEmail = "";  
35     private int OTPResendCooldown = 60;  
36     private int OTPExpirationMinutes = 5;
```

Hàm IsEmailExists(): được sử dụng để kiểm tra email người dùng có trong cơ sở dữ liệu không

conn = new NpgsqlConnection(connString): được sử dụng để khởi tạo kết nối đến cơ sở dữ liệu trên supabase thông qua chuỗi kết nối đã được khởi tạo trước đó.

query = "SELECT COUNT() FROM users WHERE email = @email":* chuỗi truy vấn đếm xem có bao nhiêu tài khoản email trùng với tài khoản của người dùng nhập.

Biến *cmd* được sử dụng để thực hiện lệnh truy vấn, dùng hàm *AddWithValue("@email", email)* để truyền tham số email của người dùng cho chuỗi *@email* trong câu truy vấn.

Kết quả của câu truy vấn sẽ được lưu trong biến *count* và kiểm tra nếu *count > 0* nghĩa là có ít nhất 1 email đã tồn tại trong cơ sở dữ liệu.

```

63 2 references
64 private bool IsEmailExists(string email)
65 {
66     try
67     {
68         using (var conn = new NpgsqlConnection(connString))
69         {
70             conn.Open();
71             string query = "SELECT COUNT(*) FROM users WHERE email = @email";
72             using (var cmd = new NpgsqlCommand(query, conn))
73             {
74                 cmd.Parameters.AddWithValue("@email", email);
75                 long count = (long)cmd.ExecuteScalar();
76                 return count > 0;
77             }
78         }
79     }
80     catch (NpgsqlException ex)
81     {
82         lblCheck.Text = $"Lỗi kết nối: {ex.Message}";
83         return false;
84     }
85 }

```

Hàm SendOTPEmail(): dùng để gửi mã OTP qua tài khoản email của người dùng

smtpClient = new SmtplibClient("smtp.gmail.com"); dùng để gửi email thông qua giao thức SMTP (Simple Mail Transfer Protocol) với máy server là "smtp.gmail.com"

- *Port = 587:* cổng SMTP của Gmail (hỗ trợ giao thức TLS/STARTTLS).
- *Credentials = new NetworkCredential(SmtplibUsername, SmtplibPassword);* sử dụng tài khoản (sona.feelthemusic@gmail.com) dùng để gửi email cho người dùng
- *EnableSsl = true:* bật giao thức bảo mật TLS/SSL để mã hóa kết nối, đảm bảo an toàn khi gửi email.

mailMessage được sử dụng để ghi các nội dung cần gửi, sau đó sử dụng phương thức bất đồng bộ *await smtpClient.SendMailAsync(mailMessage)* để gửi email với thông điệp đó đến tài khoản người dùng

```

86 // Hàm gửi email chứa mã OTP
87 2 references
88 private async Task<bool> SendOTPEmail(string email, string otp)
89 {
90     try
91     {
92         var smtpClient = new SmtpClient("smtp.gmail.com")
93         {
94             Port = 587,
95             Credentials = new NetworkCredential(SmtpUsername, SmtpPassword),
96             EnableSsl = true,
97         };
98
99         var mailMessage = new MailMessage
100         {
101             From = new MailAddress(SmtpUsername),
102             Subject = $"Mã OTP xác nhận đăng ký tài khoản SONA",
103             Body = $"Mã OTP của bạn là: <strong>{otp}</strong><br>Vui lòng nhập mã này để hoàn tất đăng ký.",
104             IsBodyHtml = true,
105         };
106         mailMessage.To.Add(email);
107
108         await smtpClient.SendMailAsync(mailMessage);
109         return true;
110     }
111     catch (Exception)
112     {
113         return false;
114     }
115 }

```

Hàm btnNext_Click(): dùng để lấy email nhập từ người dùng, sau đó gửi mã OTP đến tài khoản gmail của họ

Kiểm tra email không rỗng, định dạng hợp lệ, và chưa tồn tại.

Kiểm tra thời gian chờ gửi lại OTP

- *if (otpStore.ContainsKey(email))*: kiểm tra xem email đã có mã OTP trong otpStore (dictionary lưu trữ OTP và thời gian tạo) chưa.
- Nếu có, lấy dữ liệu OTP (otpData) và kiểm tra thời gian trôi qua từ khi tạo OTP
 - *if ((DateTime.Now - otpData.createdTime).TotalSeconds < OTPResendCooldown)*: so sánh thời gian hiện tại với thời gian tạo OTP
 - Nếu chưa vượt quá *OTPREsendCooldown* (60 giây), tính toán thời gian chờ còn lại và hiển thị thông báo

Tạo và gửi mã OTP

- *string otp = GenerateOTP()*: gọi hàm *GenerateOTP* để tạo một mã OTP ngẫu nhiên 6 chữ số.

- *if (await SendOTPEmail(email, otp))*: gọi hàm bất đồng bộ SendOTPEmail để gửi email chứa mã OTP đến địa chỉ email.
 - Nếu thành công: lưu cặp (*otp*, *DateTime.Now*) vào *otpStore* với khóa là email, đánh dấu thời gian gửi OTP.

```

116 private async void btnNext_Click(object sender, EventArgs e)
117 {
118     string email = tbEmail.Text.Trim();
119
120     if (string.IsNullOrEmpty(email))
121     {
122         lblCheck.Text = "Vui lòng nhập email.";
123         return;
124     }
125
126     if (!IsValidEmail(email))
127     {
128         lblCheck.Text = "Địa chỉ email không hợp lệ!";
129         return;
130     }
131
132     if (IsEmailExists(email))
133     {
134         lblCheck.Text = "Tài khoản Email đã tồn tại, vui lòng đăng nhập.";
135         return;
136     }
137
138     if (otpStore.ContainsKey(email))
139     {
140         var otpData = otpStore[email];
141         if ((DateTime.Now - otpData.createdTime).TotalSeconds < OTPResendCooldown)
142         {
143             lblCheck.Text = $"Vui lòng chờ {Math.Ceiling(OTPResendCooldown - (DateTime.Now - otpData.createdTime).TotalSeconds)} giây!";
144             return;
145         }
146     }
147
148     string otp = GenerateOTP();
149     if (await SendOTPEmail(email, otp))
150     {
151         otpStore[email] = (otp, DateTime.Now);
152         lblCheck.Text = "Mã OTP đã được gửi tới Email của bạn!";
153     }
154     else
155     {
156         lblCheck.Text = "Lỗi gửi mã OTP. Vui lòng kiểm tra lại email hoặc thử lại sau!";
157     }
158 }

```

Hàm btnRefreshOTP_Click(): được sử dụng để lấy lại mã OTP khi đã hết hạn

Kiểm tra email và thời gian chờ tương tự **btnNext_Click**.

Tạo và gửi *OTP* mới, cập nhật *otpStore*.


```

159
160 1 reference
161 private async void btnRefreshOTP_Click(object sender, EventArgs e)
162 {
163     string email = tbEmail.Text.Trim();
164
165     if (string.IsNullOrEmpty(email))
166     {
167         lblCheck.Text = "Vui lòng nhập email trước khi yêu cầu OTP.";
168         return;
169     }
170
171     if (!IsValidEmail(email))
172     {
173         lblCheck.Text = "Địa chỉ email không hợp lệ!";
174         return;
175     }
176
177     if (otpStore.ContainsKey(email))
178     {
179         var otpData = otpStore[email];
180         double remainingSeconds = OTPResendCooldown - (DateTime.Now - otpData.createdTime).TotalSeconds;
181         if (remainingSeconds > 0)
182         {
183             lblCheck.Text = $"Vui lòng chờ {Math.Ceiling(remainingSeconds)} giây trước khi gửi lại!";
184             return;
185         }
186     }
187
188     string newOTP = GenerateOTP();
189     if (await SendOTPEmail(email, newOTP))
190     {
191         otpStore[email] = (newOTP, DateTime.Now);
192         lblCheck.Text = "Mã OTP mới đã được gửi tới Email của bạn!";
193     }
194     else
195     {
196         lblCheck.Text = "Lỗi gửi mã OTP. Vui lòng kiểm tra lại email hoặc thử lại sau!";
197     }
198 }

```

Hàm btnSignUp_Click(): dùng để kiểm tra lại thông tin người dùng và chuyển đến form SignUpInfo

Kiểm tra *email* và *OTP* không rỗng.

Xác minh *OTP* tồn tại, chưa hết hạn, và khớp với mã nhập.

Lưu *srcEmail* với giá trị là *email* nhập từ người dùng, xóa *OTP* của người dùng đó khỏi *otpStore*, và chuyển sang **SignUpInfo** để điền thêm thông tin đăng kí.

```

2 references
199 private void btnSignUp_Click(object sender, EventArgs e)
200 {
201     string email = tbEmail.Text.Trim();
202     string otpInput = tbOTP.Text.Trim();
203
204     if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(otpInput))
205     {
206         lblCheck.Text = "Vui lòng nhập đầy đủ email và mã OTP.";
207         return;
208     }
209
210     if (!otpStore.ContainsKey(email))
211     {
212         lblCheck.Text = "Mã OTP không tồn tại! Vui lòng yêu cầu mã mới.";
213         return;
214     }
215
216     var otpData = otpStore[email];
217     if ((DateTime.Now - otpData.createdTime).TotalMinutes > OTPExpirationMinutes)
218     {
219         otpStore.Remove(email);
220         lblCheck.Text = "Mã OTP đã hết hạn! Vui lòng yêu cầu mã mới.";
221         return;
222     }
223
224     if (otpInput != otpData.otp)
225     {
226         lblCheck.Text = "Mã OTP không chính xác!";
227         return;
228     }
229
230     srcEmail = email;
231     otpStore.Remove(email);
232
233     SignUpInfo signUpInfo = new SignUpInfo(main, srcEmail);
234     main.pnLogin.Controls.Clear();
235     main.pnLogin.Controls.Add(signUpInfo);
236 }

```

Hàm btnSignUpGoogle_Click(): phương thức đăng kí bằng tài khoản Google của người dùng

Sử dụng *GoogleWebAuthorizationBroker* để xác thực với *Client ID* và *Client Secret*.

Lấy thông tin email từ tài khoản Google sau khi xác thực thành công.

Nếu email chưa tồn tại, chuyển sang **SignUpInfo**; nếu đã tồn tại, hiển thị thông báo đăng nhập.

```

238 1 reference
239 private async void btnSignUpGoogle_Click(object sender, EventArgs e)
240 {
241     try
242     {
243         string credPath = Path.Combine(Path.GetTempPath(), Path.GetRandomFileName());
244         Directory.CreateDirectory(credPath);
245
246         var clientSecrets = new ClientSecrets
247         {
248             ClientId = "266768311409-sa0qg8353t75tscss8c71v44usk0cimq.apps.googleusercontent.com",
249             ClientSecret = "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"
250         };
251
252         var scopes = new[] { "profile", "email" };
253
254         var credential = await GoogleWebAuthorizationBroker.AuthorizeAsync(
255             clientSecrets,
256             scopes,
257             "user",
258             CancellationToken.None,
259             new FileDataStore(credPath, true)
260         );
261
262         if (credential != null && credential.Token != null)
263         {
264             var oauthService = new Oauth2Service(new BaseClientService.Initializer()
265             {
266                 HttpClientInitializer = credential
267             });
268
269             Userinfo userInfo = await oauthService.Userinfo.Get().ExecuteAsync();
270             srcEmail = userInfo.Email;
271
272             if (!IsEmailExists(srcEmail))
273             {
274                 main.Activate();
275
276                 SignUpInfo signUpInfo = new SignUpInfo(main, srcEmail);
277                 main.pnLogin.Controls.Clear();
278                 main.pnLogin.Controls.Add(signUpInfo);
279             }
280         }
281     }
282     catch { }
283 }

```

SignUpInfo.cs: gửi các thông tin nhập từ người dùng lên cơ sở dữ liệu để hoàn tất đăng kí

Hàm IsPhoneNumberExists(): dùng để kiểm tra xem số điện thoại nhập từ người dùng đã xuất hiện trong cơ sở dữ liệu không

```

30
31 1 reference
32 private bool IsPhoneNumberExists(string phoneNumber)
33 {
34     try
35     {
36         using (var conn = new NpgsqlConnection(connString))
37         {
38             conn.Open();
39             string query = "SELECT COUNT(*) FROM users WHERE phone_number = @phone_number";
40             using (var cmd = new NpgsqlCommand(query, conn))
41             {
42                 cmd.Parameters.AddWithValue("@phone_number", phoneNumber);
43                 long count = (long)cmd.ExecuteScalar();
44                 return count > 0;
45             }
46         }
47     }
48     catch (NpgsqlException ex)
49     {
50         lblCheckSdt.Text = $"Lỗi kiểm tra số điện thoại: {ex.Message}";
51         return false;
52     }

```

Hàm checkSignUpInfor(): được sử dụng để kiểm tra trường thông tin nhập từ người dùng

Tên người dùng (*tbUser.Text*):

- Nếu rỗng, hiển thị lỗi "*Tên người dùng không được để trống!*" và trả về *false*.

Số điện thoại (*tbSdt.Text*):

- Nếu rỗng, hiển thị lỗi "*Vui lòng nhập số điện thoại!*".
- Nếu chứa ký tự không phải số, hiển thị lỗi "*Số điện thoại không hợp lệ!*".
- Nếu số điện thoại đã tồn tại (gọi *IsPhoneNumberExists*), hiển thị lỗi "*Số điện thoại đã tồn tại!*".

Mật khẩu (*tbPass.Text*):

- Nếu rỗng trả về *false*.
- Kiểm tra mật khẩu có ít nhất: một chữ số (*checkNum*), một chữ cái (*checkLetter*), một ký tự đặc biệt (@, #, !, hoặc ?) (*checkSpecial*). Nếu không đủ 1 trong 3 điều kiện, trả về *false*.

Xác nhận mật khẩu (*tbConfirm.Text*):

- Nếu rỗng, hiển thị lỗi "*Vui lòng xác nhận mật khẩu!*".
- Nếu không khớp với mật khẩu, hiển thị lỗi "*Mật khẩu nhập lại chưa chính xác!*".

Trả về: *true* nếu tất cả thông tin hợp lệ, *false* nếu có lỗi.

```
54 // Hàm kiểm tra thông tin đăng nhập và báo lỗi nếu không hợp lệ
55 1 reference
56 private bool checkSignUpInfor()
57 {
58     lblCheckName.Text = lblCheckSdt.Text = lblCheckConfirm.Text = "";
59     lblcheckPass.ForeColor = Color.FromArgb(102, 102, 102);
60
61     if (string.IsNullOrEmpty(tbUser.Text))
62     {
63         lblCheckName.Text = "Tên người dùng không được để trống!";
64         return false;
65     }
66
67     if (string.IsNullOrEmpty(tbSdt.Text))
68     {
69         lblCheckSdt.Text = "Vui lòng nhập số điện thoại!";
70         return false;
71     }
72
73     for (int i = 0; i < tbSdt.Text.Length; i++)
74     {
75         if (!char.IsDigit(tbSdt.Text[i]))
76         {
77             lblCheckSdt.Text = "Số điện thoại không hợp lệ!";
78             return false;
79         }
80     }
81
82     if (IsPhoneNumberExists(tbSdt.Text))
83     {
84         lblCheckSdt.Text = "Số điện thoại đã tồn tại!";
85         return false;
86     }
87
88     if (string.IsNullOrEmpty(tbPass.Text))
89     {
90         lblcheckPass.ForeColor = Color.Red;
91         return false;
92     }
93 }
```

Hàm btnSignUp_Click(): kiểm tra và lưu thông tin của người dùng vào cơ sở dữ liệu.

Gọi **checkSignUpInfor()** để kiểm tra thông tin nhập vào.

Mở kết nối đến cơ sở dữ liệu với *NpgsqlConnection*.

- Thực hiện truy vấn SQL INSERT để thêm người dùng mới vào bảng users với các giá trị: *name_user*, *email*, *phone_number*, và *password*.
- Sử dụng tham số hóa (*@name_user*, *@email*, v.v.) để tránh truyền giá trị nhập từ người dùng vào câu truy vấn.

Gọi **main.ShowHome(srcEmail)** để chuyển đến màn hình chính với email người dùng.

Nếu có lỗi (ví dụ: mất kết nối, vi phạm ràng buộc cơ sở dữ liệu), hiển thị thông báo lỗi trên *lblCheckName*.

```

129 5 references
130 private void btnSignUp_Click(object sender, EventArgs e)
131 {
132     if (checkSignUpInfor())
133     {
134         try
135         {
136             using (var conn = new NpgsqlConnection(connString))
137             {
138                 conn.Open();
139                 string query = "INSERT INTO users (name_user, email, phone_number, password) VALUES
140 using (var cmd = new NpgsqlCommand(query, conn))
141             {
142                 cmd.Parameters.AddWithValue("@name_user", tbUser.Text.Trim());
143                 cmd.Parameters.AddWithValue("@email", srcEmail);
144                 cmd.Parameters.AddWithValue("@phone_number", tbSdt.Text.Trim());
145                 cmd.Parameters.AddWithValue("@password", tbPass.Text.Trim());
146
147                 cmd.ExecuteNonQuery();
148                 main.ShowHome(srcEmail);
149             }
150         }
151     }
152     catch (Exception ex)
153     {
154         lblCheckName.Text = "Lỗi thêm người dùng vào dữ liệu: " + ex.Message;
155     }
156 }
157

```

Login.cs: cung cấp phương thức đăng nhập bằng tài khoản mật khẩu hoặc Google cho người dùng

Hàm btnDangNhap_Click(): thực hiện xác thực tài khoản mật khẩu nhập từ người dùng và chuyển đến form Home

```

53 1 reference
54 private void btnDangNhap_Click(object sender, EventArgs e)
55 {
56     string email = tbEmail.Text.Trim();
57     string password = tbPass.Text.Trim();
58
59     if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password))
60     {
61         lblCheck.Text = "Vui lòng nhập đầy đủ email và mật khẩu.";
62         return;
63     }
64
65     try
66     {
67         using (var conn = new NpgsqlConnection(connString))
68         {
69             conn.Open();
70             string query = "SELECT id_user, email FROM users WHERE email = @email AND password = @password";
71             using (var cmd = new NpgsqlCommand(query, conn))
72             {
73                 cmd.Parameters.AddWithValue("@email", email);
74                 cmd.Parameters.AddWithValue("@password", password);
75                 using (var reader = cmd.ExecuteReader())
76                 {
77                     if (reader.Read())
78                     {
79                         srcEmail = reader["email"].ToString();
80                         main.ShowHome(srcEmail);
81                     }
82                     else
83                     {
84                         lblCheck.Text = "Email hoặc mật khẩu không đúng.";
85                     }
86                 }
87             }
88         }
89     }
90     catch (NpgsqlException ex)
91     {
92         lblCheck.Text = $"Lỗi kết nối: {ex.Message}";
93     }
94 }

```

Lấy dữ liệu đầu vào

- Lấy email và mật khẩu từ các trường nhập liệu *tbEmail* và *tbPass* (TextBox).
- Sử dụng *Trim()* để loại bỏ khoảng trắng đầu và cuối, đảm bảo dữ liệu sạch.

if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password)) kiểm tra xem email hoặc mật khẩu có rỗng không. Nếu có hiển thị thông báo lỗi và *return*.

Xác thực thông tin đăng nhập

- *conn = new NpgsqlConnection(connString)*: kết nối đến cơ sở dữ liệu Supabase với chuỗi kết nối *connString*, sử dụng *conn.Open()* để mở kết nối.
- *string query = "SELECT id_user, email FROM users WHERE email = @email AND password = @password"*: truy vấn này tìm kiếm trong bảng *users* một bản ghi có

email và password khớp với giá trị người dùng nhập. Lấy cột *id_user* và *email* nếu tìm thấy

- Sử dụng tham số hóa (*@email*, *@password*) để truyền giá trị vào, tránh trường hợp SQL Injection.

Thực thi truy vấn bằng `cmd.ExecuteReader()` và sử dụng biến reader để lấy các bản ghi

- `if (reader.Read())`: kiểm tra xem có bản ghi nào khớp không.
- Nếu có:
 - `srcEmail = reader["email"].ToString()`: lưu email vào biến `srcEmail`.
 - `main.ShowHome(srcEmail)`: chuyển đến màn hình chính (Home) với email người dùng.

Hàm `btnLoginGoogle_Click()`: phương thức đăng nhập thông qua tài khoản Google của người dùng

- Tạo thư mục tạm (`credPath`) để lưu thông tin xác thực Google.
- Cấu hình `ClientSecrets` với `ClientId` và `ClientSecret` của ứng dụng Google.
- Yêu cầu quyền truy cập (`scopes: "profile", "email"`) và xác thực qua `GoogleWebAuthorizationBroker`.
- Nếu xác thực thành công:
 - Sử dụng `Oauth2Service` để lấy thông tin người dùng (`Userinfo`).
 - Lưu email vào `srcEmail`.
 - Kiểm tra email tồn tại trong cơ sở dữ liệu (`IsEmailExists`).
 - Nếu tồn tại, gọi **`main.ShowHome(userEmail)`** để chuyển sang màn hình chính.
 - Nếu không tồn tại, hiển thị thông báo "Email chưa tồn tại, vui lòng đăng kí tài khoản!".


```

1 reference
99 private async void btnLoginGoogle_Click(object sender, EventArgs e)
100 {
101     try
102     {
103         string credPath = Path.Combine(Path.GetTempPath(), Path.GetRandomFileName());
104         Directory.CreateDirectory(credPath);
105
106         var clientSecrets = new ClientSecrets
107         {
108             ClientId = "266768311409-sa0qg8353t75tscss8c71v44usk0cimq.apps.googleusercontent.com",
109             ClientSecret = "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"
110         };
111
112         var scopes = new[] { "profile", "email" };
113
114         var credential = await GoogleWebAuthorizationBroker.AuthorizeAsync(
115             clientSecrets,
116             scopes,
117             "user",
118             CancellationToken.None,
119             new FileDataStore(credPath, true)
120         );
121
122         if (credential != null && credential.Token != null)
123         {
124             var oauthService = new OAuth2Service(new BaseClientService.Initializer()
125             {
126                 HttpClientInitializer = credential
127             });
128
129             Userinfo userInfo = await oauthService.Userinfo.Get().ExecuteAsync();
130             srcEmail = userInfo.Email;
131
132             if (IsEmailExists(srcEmail))
133             {
134                 main.Activate();
135                 main.ShowHome(srcEmail);
136             }
137             else
138             {
139                 main.Activate();
140                 lblCheck.Text = "Email chưa tồn tại, vui lòng đăng kí tài khoản!";
141             }
142         }
143     }
144     catch { }
145 }

```

ForgetPassword.cs: cung cấp phương thức đổi mật khẩu cho người dùng

Xóa các thông báo lỗi trước đó.

Kiểm tra email và OTP không rỗng.

Xác thực OTP:

- Kiểm tra OTP tồn tại, chưa hết hạn, và khớp với mã nhập.

Kiểm tra định dạng mật khẩu mới và xác nhận mật khẩu.

Cập nhật mật khẩu trong cơ sở dữ liệu:

- Nếu thành công, xóa OTP và chuyển về form Login.
- Nếu thất bại, hiển thị lỗi.

```

260 4 references
261 private void btnConfirm_Click(object sender, EventArgs e)
262 {
263     lblCheckEmail.Text = lblCheckOTP.Text = lblCheckConfirmPass.Text = "";
264     lblCheckPass.ForeColor = Color.FromArgb(102, 102, 102);
265
266     string email = tbEmail.Text.Trim();
267     string otpInput = tbOTP.Text.Trim();
268
269     if (string.IsNullOrEmpty(email))
270     {
271         lblCheckEmail.Text = "Vui lòng nhập địa chỉ Email!";
272         return;
273     }
274
275     if (string.IsNullOrEmpty(otpInput))
276     {
277         lblCheckOTP.Text = "Vui lòng nhập mã OTP!";
278         return; // Sửa: Thêm return để dừng nếu OTP trống
279     }
280
281     if (!otpStore.ContainsKey(email))
282     {
283         lblCheckOTP.Text = "Mã OTP không tồn tại! Vui lòng yêu cầu mã mới.";
284         return;
285     }
286
287     var otpData = otpStore[email];
288     if ((DateTime.Now - otpData.createdTime).TotalMinutes > OTPExpirationMinutes)
289     {
290         otpStore.Remove(email);
291         lblCheckOTP.Text = "Mã OTP đã hết hạn! Vui lòng yêu cầu mã mới.";
292         return;
293     }
294
295     if (otpInput != otpData.otp)
296     {
297         lblCheckOTP.Text = "Mã OTP không chính xác!";
298         return;
299     }

```

```

299
300     if (!checkPassFormat()) return;
301
302     try
303     {
304         using (var conn = new NpgsqlConnection(connString))
305         {
306             conn.Open();
307             string query = "UPDATE users SET password = @password WHERE email = @email";
308             using (var cmd = new NpgsqlCommand(query, conn))
309             {
310                 cmd.Parameters.AddWithValue("@password", tbPass.Text);
311                 cmd.Parameters.AddWithValue("@email", email);
312                 int rowsAffected = cmd.ExecuteNonQuery();
313
314                 if (rowsAffected > 0)
315                 {
316                     otpStore.Remove(email); // Xóa OTP sau khi cập nhật thành công
317
318                     Login login = new Login(main);
319                     main.pnLogin.Controls.Clear();
320                     main.pnLogin.Controls.Add(login); // Chuyển về form Login
321                 }
322                 else
323                 {
324                     lblCheckEmail.Text = "Không tìm thấy tài khoản để đổi mật khẩu.";
325                 }
326             }
327         }
328     }

```

HomeContent.cs: hiển thị dữ liệu và trạng thái lên giao diện

Khởi tạo các biến toàn cục

- *userEmail*: email của người dùng hiện tại.
- *idUser*, *channelId*, *readApiKey*, *deviceName*: thông tin thiết bị và người dùng từ cơ sở dữ liệu.
- *connString*: chuỗi kết nối đến Supabase.
- *smtpUsername*, *smtpPassword*: thông tin tài khoản Gmail để gửi email.
- *tempAlertSent*, *lastTempAlertTime*, *alertIntervalMinutes*: quản lý gửi email cảnh báo nhiệt độ khi vượt ngưỡng (5 phút/lần).
- *refreshInterval*: thời gian làm mới dữ liệu (30 giây).
- *espOfflineThreshold*: ngưỡng thời gian để coi ESP8266 là offline (5 phút).
- *httpClient*: client để gửi yêu cầu HTTP đến ThingSpeak.
- *isFirstUpdate*: cờ kiểm soát lần cập nhật dữ liệu lần đầu tiên.
- *chatBot*: đối tượng dùng để gọi và gửi dữ liệu đến usercontrol Chatbot.cs

```
23 private string userEmail;  
24 private string idUser;  
25 private string channelId;  
26 private string readApiKey;  
27 private string deviceName;  
28  
29 private const string connString = "Host=aws-0-ap-southeast-1.pooler.supabase.com;Port=5432;Database=postgres";  
30  
31 // Cấu hình email gửi thông báo  
32 private const string smtpUsername = "sona.feelthemusic@gmail.com";  
33 private const string smtpPassword = "sxiz nqtc ddko izzj";  
34  
35 private bool temperatureAlertSent = false; // Cờ để kiểm soát gửi email nhiệt độ lần đầu  
36 private DateTime? lastTemperatureAlertTime = null; // Thời gian gửi email cảnh báo nhiệt độ cuối cùng  
37 private const int alertIntervalMinutes = 5; // Gửi email định kỳ mỗi 5 phút  
38  
39 private int refreshInterval = 30; // Thời gian làm mới dữ liệu là 30 giây  
40 private int espOfflineThreshold = 5 * 60; // Thời gian tối đa để coi ESP8266 là offline 5 phút  
41 private HttpClient httpClient = new HttpClient();  
42  
43 private bool isFirstUpdate = false; // Cờ kiểm soát lần cập nhật đầu tiên  
44 private ChatBot chatBot = new ChatBot();  
45
```

Hàm InitializeUserAndDevice(): lấy thông tin người dùng và thiết bị thông qua email của họ

Kết nối đến Supabase thông qua chuỗi kết nối.

Thực hiện truy vấn “*SELECT id_user FROM users WHERE email = @email*” để lấy *id_user* bằng việc kiểm tra thuộc tính email của người dùng.

Dựa vào `id_user` của người dùng ta đã lấy được, tiếp tục thực hiện lệnh truy vấn “*SELECT device_name, channel_id, api_key FROM devices WHERE id_user = @id_user*” để lấy các thông tin cần thiết của thiết bị của người dùng đó.

Gán các giá trị đã lấy được cho các biến cục bộ để sử dụng trong việc gọi API ThingSpeak và hiển thị dữ liệu cho các hàm phía dưới.

Hiển thị thông báo lỗi nếu không tìm thấy dữ liệu hoặc có ngoại lệ.

```
77 // Khởi tạo thông tin người dùng và thiết bị từ cơ sở dữ liệu
78 1 reference
79 private void InitializeUserAndDevice()
80 {
81     try
82     {
83         using (var conn = new NpgsqlConnection(connString))
84         {
85             conn.Open();
86             // Lấy id_user từ bảng users
87             string userQuery = "SELECT id_user FROM users WHERE email = @email";
88             using (var userCmd = new NpgsqlCommand(userQuery, conn))
89             {
90                 userCmd.Parameters.AddWithValue("@email", userEmail);
91                 using (var reader = userCmd.ExecuteReader())
92                 {
93                     if (reader.Read())
94                     {
95                         idUser = reader["id_user"].ToString();
96                     }
97                     else
98                     {
99                         MessageBox.Show("Không tìm thấy thông tin người dùng!");
100                         return;
101                     }
102                 }
103             }
104         }
105     }
106     // Lấy channel_id và readApiKey từ bảng devices
107     string deviceQuery = "SELECT device_name, channel_id, api_key FROM devices WHERE id_user = @id_user";
108     using (var deviceCmd = new NpgsqlCommand(deviceQuery, conn))
109     {
110         deviceCmd.Parameters.AddWithValue("@id_user", int.Parse(idUser));
111         using (var reader = deviceCmd.ExecuteReader())
112         {
113             if (reader.Read())
114             {
115                 deviceName = reader["device_name"].ToString();
116                 channelId = reader["channel_id"].ToString();
117                 readApiKey = reader["api_key"].ToString();
118             }
119             else
120             {
121                 channelId = null;
122                 readApiKey = null;
123                 return;
124             }
125         }
126     }
127 }
```

Hàm FetchData(): lấy dữ liệu từ ThingSpeak

Kiểm tra trường thông tin *channelID* và *readApiKey* trước khi thực hiện.

Gửi yêu cầu HTTP và sử dụng *GetStringAsync* để lấy dữ liệu JSON từ URL.

JObject.Parse chuyển chuỗi JSON thành đối tượng *JObject* để xử lý.

```
134
135 // Hàm lấy dữ liệu từ ThingSpeak
136 2 references
137 private async Task<JObject> FetchData(string url)
138 {
139     if (string.IsNullOrEmpty(channelId) || string.IsNullOrEmpty(readApiKey))
140     {
141         lblStatus.Text = "Không có thông tin API hoặc Channel ID!";
142         return null;
143     }
144     try
145     {
146         var response = await httpClient.GetStringAsync(url);
147         return JObject.Parse(response);
148     }
149     catch
150     {
151         return null;
152     }
153 }
```

Hàm SendAlertEmail(): gửi email cảnh báo đến người dùng

Cấu hình *SmtpClient* dùng để gửi tin nhắn qua ứng dụng Gmail.

Tạo *MailMessage* với tiêu đề và thông điệp cần gửi.

Sử dụng phương thức bất đồng bộ để gửi email, và xử lý các lỗi ngoại lệ.

```
154
155 // Hàm gửi email thông báo
156 1 reference
157 private async Task SendAlertEmail(string subject, string body)
158 {
159     try
160     {
161         using (var smtpClient = new SmtpClient("smtp.gmail.com"))
162         {
163             smtpClient.Port = 587;
164             smtpClient.Credentials = new NetworkCredential(smtpUsername, smtpPassword);
165             smtpClient.EnableSsl = true;
166             var mailMessage = new MailMessage
167             {
168                 From = new MailAddress(smtpUsername),
169                 Subject = subject,
170                 Body = body,
171                 IsBodyHtml = true,
172             };
173             mailMessage.To.Add(userEmail);
174             await smtpClient.SendMailAsync(mailMessage);
175         }
176     }
177 }
```

Hàm UpdateRefreshTime(): cập nhật thời gian và trạng thái

Chuyển đổi thời gian: Chuyển *createdAt* (chuỗi từ ThingSpeak) sang giờ Việt Nam.

So sánh chênh lệch thời gian (*timeDiff*) với *espOfflineThreshold* để xác định ESP8266 online/offline.

Cập nhật giao diện *lblStatus* (trạng thái) và *lblRefreshData* (thời gian) từ dữ liệu được ThingSpeak gửi tới.

```
184 // Hàm cập nhật thời gian làm mới dữ liệu
185 1 reference
186 private void UpdateRefreshTime(string createdAt, DateTime vietnamTime)
187 {
188     if (createdAt != null)
189     {
190         try
191         {
192             var createdAtUtc = DateTime.Parse(createdAt, null, System.Globalization.DateTimeStyles.RoundtripKind);
193             if (createdAtUtc.Kind != DateTimeKind.Utc)
194             {
195                 createdAtUtc = DateTime.SpecifyKind(createdAtUtc, DateTimeKind.Utc);
196             }
197             var vietnamTimeZone = TimeZoneInfo.FindSystemTimeZoneById("SE Asia Standard Time");
198             var createdAtVietnam = TimeZoneInfo.ConvertTimeFromUtc(createdAtUtc, vietnamTimeZone);
199             var timeDiff = (DateTime.UtcNow - createdAtUtc).TotalSeconds;
200             lblStatus.Text = timeDiff <= espOfflineThreshold
201                 ? $"Trạng thái {deviceName}: Online"
202                 : $"Trạng thái {deviceName}: Offline";
203             lblRefreshData.Text = $"Dữ liệu được làm mới lúc: {createdAtVietnam:yyyy-MM-dd hh:mm:ss tt}";
204         }
205         catch (Exception ex)
206         {
207             lblStatus.Text = "Lỗi khi chuyển đổi thời gian.";
208             lblRefreshData.Text = $"Dữ liệu được làm mới lúc: {vietnamTime:yyyy-MM-dd hh:mm:ss tt} (Lỗi: {ex.Message})";
209         }
210     }
211     else
212     {
213         lblStatus.Text = $"Trạng thái {deviceName}: Không có thông tin thời gian.";
214         lblRefreshData.Text = $"Dữ liệu được làm mới lúc: {vietnamTime:yyyy-MM-dd hh:mm:ss tt} (Không có dữ liệu)";
215     }
216 }
217
218
```

Hàm UpdateData(): xử lý dữ liệu từ ThingSpeak

Các biến được sử dụng để lấy dữ liệu

- *latestData*: gọi phương thức *FetchData* để lấy 1 bản ghi mới nhất từ ThingSpeak thông qua URL.
- *currentTimeUtc*: lấy thời gian hiện tại theo UTC.
- *vietnamTime*: chuyển đổi sang múi giờ Việt Nam (SE Asia Standard Time, UTC+7)

Kiểm tra và xử lý dữ liệu mới nhất

- Kiểm tra xem dữ liệu từ ThingSpeak có hợp lệ không *latestData != null*.
- *latestData ["feeds"][0]*: truy cập bản ghi đầu tiên trong mảng feeds của dữ liệu JSON
 - *temperature*: lấy giá trị nhiệt độ từ field1.
 - *humidity*: lấy giá trị độ ẩm từ field2
 - *relayState*: lấy trạng thái relay từ field3.
 - *createdAt*: lấy thời gian tạo bản ghi từ *created_at*.

Cập nhật giao diện với dữ liệu nhận được

- *UpdateRefreshTime(createdAt, vietnamTime)*:
 - Cập nhật thời gian làm mới dữ liệu trên nhãn *lblRefreshData*.
 - Kiểm tra trạng thái ESP8266 (Online/Offline) dựa trên *createdAt* và hiển thị trên *lblStatus*.
- *UpdateRelayStatus(relayState)*:
 - Cập nhật trạng thái relay trên nút *btnRelay*.
 - Nếu *relayState* là 1, hiển thị "Bật" nếu là 0, hiển thị "Tắt", nếu không hợp lệ, hiển thị thông báo lỗi.
- *UpdateTemperatureAndHumidity(temperature, humidity)*:
 - Cập nhật nhiệt độ trên *btnTemperature* và độ ẩm trên *btnHumidity*.
 - Gửi dữ liệu cho chatBot để chatbot hiển thị thông tin.

Lấy dữ liệu lịch sử để vẽ biểu đồ

- Biến *recentData* gọi **FetchData()** để lấy 200 bản ghi lịch sử từ ThingSpeak
- Chuyển đổi dữ liệu:
 - Sử dụng LINQ để ánh xạ từng bản ghi trong *recentData["feeds"]* thành một tuple (Time, Temperature, Humidity).
- Xử lý nhiệt độ và độ ẩm:
 - *float.TryParse(f["field1"]?.ToString(), out tempValue)*: chuyển chuỗi nhiệt độ (field1) thành số thực.

- `Math.Round(tempValue, 2)`: làm tròn giá trị nhiệt độ đến 2 chữ số thập phân.
- Xử lý thời gian:
 - `createdAtStr`: lấy thời gian từ `created_at`.
 - Nếu `createdAtStr` hợp lệ, chuyển thành `DateTime` và đổi sang UTC; nếu không, sử dụng `DateTime.UtcNow`.
- Lọc dữ liệu:
 - `.Where(f => f.Temperature.HasValue || f.Humidity.HasValue)`: chỉ giữ lại các bản ghi có ít nhất một giá trị nhiệt độ hoặc độ ẩm hợp lệ.
- Kết quả feeds sẽ là danh sách các tuple chứa thời gian, nhiệt độ, và độ ẩm để vẽ biểu đồ ta gọi hàm **PlotTemperature(feeds)** và **PlotHumidity(feeds)**.
- Truyền giá trị `feeds` thông qua phương thức **UpdateHistoricalData** của Chatbot để nó nhận được các dữ liệu nhiệt độ và độ ẩm theo từng mốc thời gian.

```

221
222 // Hàm cập nhật dữ liệu từ ThingSpeak
223 1 reference
224 private async Task UpdateData()
225 {
226     if (string.IsNullOrEmpty(channelId) || string.IsNullOrEmpty(readApiKey))
227     {
228         return;
229     }
230
231     try
232     {
233         // Lấy dữ liệu mới nhất (1 bản ghi) từ ThingSpeak và chuyển đổi giờ UTC sang giờ Việt Nam
234         var latestData = await FetchData($"https://api.thingspeak.com/channels/{channelId}/feeds.json?api_key={readApi
235         var currentTimeUtc = DateTime.UtcNow;
236         var vietnamTimeZone = TimeZoneInfo.FindSystemTimeZoneById("SE Asia Standard Time");
237         var vietnamTime = TimeZoneInfo.ConvertTimeFromUtc(currentTimeUtc, vietnamTimeZone);
238
239         // Trích xuất các thông tin từ dữ liệu từ file JSON nhận được
240         if (latestData != null)
241         {
242             var feed = latestData["feeds"][0];
243             string temperature = feed["field1"]?.ToString() ?? "Không có dữ liệu";
244             string humidity = feed["field2"]?.ToString() ?? "Không có dữ liệu";
245             string relayState = feed["field3"]?.ToString();
246             string createdAt = feed["created_at"]?.ToString();
247
248             UpdateRefreshTime(createdAt, vietnamTime); // Cập nhật thời gian làm mới dữ liệu
249             UpdateRelayStatus(relayState); // Cập nhật trạng thái Relay
250             UpdateTemperatureAndHumidity(temperature, humidity); // Cập nhật nhiệt độ và độ ẩm

```



```

251 // Lấy dữ liệu lịch sử để vẽ biểu đồ và truyền cho ChatBot (200 bản ghi)
252 var recentData = await FetchData($"https://api.thingspeak.com/channels/{channelId}/feeds.json?api_key={readApiKey}")
253 if (recentData != null)
254 {
255     var feeds = recentData["feeds"]
256     .Select(f =>
257     {
258         float tempValue, humValue;
259         float? temp = float.TryParse(f["field1"].ToString(), out tempValue) ? (float?)Math.Round(tempValue, 2) :
260         float? hum = float.TryParse(f["field2"].ToString(), out humValue) ? (float?)Math.Round(humValue, 2) :
261
262         string createdAtStr = f["created_at"].ToString();
263         DateTime time;
264         if (!string.IsNullOrEmpty(createdAtStr))
265         {
266             time = DateTime.Parse(createdAtStr, null, System.Globalization.DateTimeStyles.RoundtripKind);
267             if (time.Kind != DateTimeKind.Utc)
268             {
269                 time = DateTime.SpecifyKind(time, DateTimeKind.Utc);
270             }
271             time = TimeZoneInfo.ConvertTimeFromUtc(time, vietnamTimeZone); // Chuyển đổi sang giờ Việt Nam
272         }
273         else
274         {
275             time = vietnamTime; // Sử dụng thời gian hiện tại đã chuyển đổi
276         }
277
278         return (
279             Time: time,
280             Temperature: temp,
281             Humidity: hum
282         );
283     })
284     .Where(f => f.Temperature.HasValue || f.Humidity.HasValue)
285     .ToList();
286
287     // Vẽ biểu đồ nhiệt độ và độ ẩm
288     PlotTemperature(feeds);
289     PlotHumidity(feeds);
290
291     // Truyền dữ liệu lịch sử cho ChatBot
292     chatBot.UpdateHistoricalData(feeds);
293 }

```

Hàm UpdateTemperatureAndHumidity(): dùng để hiển thị nhiệt độ và độ ẩm lên giao diện, đồng thời gửi cảnh báo nếu nhiệt độ vượt ngưỡng

Nhận dữ liệu nhiệt độ và độ ẩm từ hàm **UpdateData()** trước đó, sau đó chuyển đổi kiểu *float* dùng để chỉ lấy số đầu tiên sau dấu phẩy và hiển thị lên giao diện

Thực hiện so sánh nhiệt độ hiện tại với mốc vượt ngưỡng, nếu vượt mốc đó thì thực hiện gọi hàm **SendAlertEmail()** để gửi thông báo vượt ngưỡng đến người dùng

Để gửi thông báo định kì mỗi 5 phút một lần, ta sử dụng cờ *tempAlertSent = true* để đánh dấu lần gửi đầu tiên thành công và dùng biến *lastTempAlertTime* để cập nhật thời gian gửi

Ở lần gửi tiếp theo nó sẽ kiểm tra điều kiện rằng, nếu chưa gửi hoặc lần gửi tiếp theo đã vượt quá mốc thời gian đánh dấu là 5 phút thì mới gửi lại cảnh báo

Nếu nhiệt độ không vượt ngưỡng, ta thực hiện đặt lại cờ và thời gian gửi tin nhắn

Cập nhật giá trị nhiệt độ và độ ẩm hiện tại cho ChatBot thông qua phương thức **UpdateTemperatureAndHumidity()**

```
341 // Hàm cập nhật nhiệt độ và độ ẩm
342 1 reference
343 private void UpdateTemperatureAndHumidity(string temperature, string humidity)
344 {
345     float tempValue = 0, humValue = 0;
346
347     // Chuyển đổi giá trị nhiệt độ và độ ẩm từ chuỗi sang số thực và hiển thị trên giao diện
348     if (temperature != "Không có dữ liệu" && float.TryParse(temperature, out tempValue))
349         btnTemperature.Text = $"{Math.Round(tempValue, 2)}°C";
350     else
351         btnTemperature.Text = "Không có dữ liệu";
352
353     if (humidity != "Không có dữ liệu" && float.TryParse(humidity, out humValue))
354         btnHumidity.Text = $"{Math.Round(humValue, 2)}%";
355     else
356         btnHumidity.Text = "Không có dữ liệu";
357
358     // Kiểm tra ngưỡng và gửi email nếu vượt ngưỡng
359     if (tempValue > 30) // Ngưỡng nhiệt độ: > 30°C
360     {
361         // Nếu chưa gửi email lần nào hoặc đã đủ 5 phút kể từ lần gửi cuối
362         if (!temperatureAlertSent || (lastTemperatureAlertTime.HasValue && (DateTime.Now - lastTemperatureAlertTime.Value
363             {
364                 string subject = $"Cảnh báo: Nhiệt độ vượt ngưỡng từ {deviceName}";
365                 string body = $"<h3>Cảnh báo từ hệ thống giám sát!</h3><p>Nhiệt độ hiện tại là {Math.Round(tempValue, 2)}°C,
366                 SendAlertEmail(subject, body);
367                 temperatureAlertSent = true; // Đánh dấu đã gửi email
368                 lastTemperatureAlertTime = DateTime.Now; // Cập nhật thời gian gửi email cuối cùng
369             }
370         }
371     }
372     else if (tempValue <= 30 && temperatureAlertSent)
373     {
374         temperatureAlertSent = false; // Đặt lại cờ khi nhiệt độ trở lại bình thường
375         lastTemperatureAlertTime = null; // Đặt lại thời gian gửi email
376     }
377     chatBot.UpdateTemperatureAndHumidity(temperature, humidity); // Cập nhật dữ liệu cho ChatBot
378 }
```

Hàm PlotTemperatue(): vẽ biểu đồ nhiệt độ dựa trên danh sách dữ liệu feeds chứa các bản ghi (Time, Temperature, Humidity).

Tạo mô hình biểu đồ

- *var model = new PlotModel:* lớp OxyPlot đại diện cho toàn bộ biểu đồ, được sử dụng để chứa các trục (axes), chuỗi dữ liệu (series) và các thiết lập trên đó.
- *{ Title = "Biểu đồ Nhiệt độ (Giờ Việt Nam)" }:* tiêu đề của biểu đồ.

Tạo và cấu trúc hình trục thời gian (X-Axis)

- *Position = AxisPosition.Bottom:* đặt trục thời gian ở phía dưới biểu đồ (trục X).
- *Title = "Thời gian":* đặt nhãn cho trục, hiển thị chữ "Thời gian" bên dưới trục.
- *IntervalType = DateTimeIntervalType.Auto:* OxyPlot sẽ tự động chọn khoảng cách giữa các nhãn thời gian (ví dụ: theo giây, phút, giờ, ngày, tùy vào phạm vi dữ liệu).

- *MajorGridlineStyle = LineStyle.Solid*: đường lưới chính (các đường dọc tương ứng với các mốc thời gian lớn) sẽ là đường liền.
- *MinorGridlineStyle = LineStyle.Dot*: đường lưới phụ (các mốc thời gian nhỏ hơn) sẽ là đường chấm.

Gắn sự kiện **DateAxis_AxisChanged** để định dạng thời gian trên trục và thêm trục thời gian vào mô hình *model.Axes.Add(dateAxis)*

Tạo và cấu hình trục nhiệt độ (Y-Axis)

- *Position = AxisPosition.Left*: đặt trục nhiệt độ ở bên trái biểu đồ (trục Y).
- *Title = "Nhiệt độ (°C)"*: đặt nhãn cho trục, hiển thị chữ "Nhiệt độ (°C)" bên cạnh trục.
- *Minimum = 0*: giá trị nhỏ nhất trên trục Y là 0°C.
- *Maximum = 50*: giá trị lớn nhất trên trục Y là 50°C.

Thêm dữ liệu nhiệt độ và độ ẩm vào chuỗi

- *feed.Temperature.HasValue*: kiểm tra xem giá trị nhiệt độ (Temperature) có hợp lệ không (float?)
- *DateTimeAxis.ToDouble(feed.Time)*: chuyển đổi thời gian (feed.Time, kiểu DateTime) thành một giá trị số kiểu double mà DateTimeAxis có thể sử dụng.
- *tempSeries.Points.Add(new DataPoint(xValue, feed.Temperature.Value))*: thêm một điểm dữ liệu vào chuỗi tempSeries với tham số
 - *xValue*: giá trị trên trục X (thời gian, đã chuyển thành double).
 - *feed.Temperature.Value*: giá trị trên trục Y (nhiệt độ, kiểu float).

Thêm chuỗi vào mô hình đồ thị *model.Series.Add(tempSeries)*, và gán model vào *pvTemperature.Model* để cập nhật biểu đồ trên giao diện người dùng

```

253 // Hàm vẽ biểu đồ nhiệt độ
254 1 reference
255 private void PlotTemperature(List<DateTime Time, float? Temperature, float? Humidity> feeds)
256 {
257     var model = new PlotModel { Title = "Biểu đồ Nhiệt độ (Giờ Việt Nam)" };
258
259     // Tạo trục thời gian
260     var dateAxis = new DateTimeAxis
261     {
262         Position = AxisPosition.Bottom, // Vị trí trục nằm ở dưới
263         Title = "Thời gian",
264         IntervalType = DateTimeIntervalType.Auto, // Tự động điều chỉnh khoảng cách giữa các nhãn
265         MajorGridLineStyle = LineStyle.Solid, // Đường lưới chính
266         MinorGridLineStyle = LineStyle.Dot // Đường lưới phụ
267     };
268
269     dateAxis.AxisChanged += DateAxis_AxisChanged; // Thêm sự kiện thay đổi trục thời gian
270     model.Axes.Add(dateAxis); // Thêm trục thời gian vào mô hình
271
272     model.Axes.Add(new LinearAxis
273     {
274         Position = AxisPosition.Left, // Vị trí trục nằm ở bên trái
275         Title = "Nhiệt độ (°C)",
276         Minimum = 0, // Giá trị tối thiểu của trục
277         Maximum = 50 // Giá trị tối đa của trục
278     });
279
280     var tempSeries = new LineSeries
281     {
282         Title = "Nhiệt độ",
283         Color = OxyColors.Orange
284     };
285
286     // Thêm các điểm dữ liệu vào biểu đồ
287     foreach (var feed in feeds)
288     {
289         if (feed.Temperature.HasValue) // Kiểm tra xem nhiệt độ có giá trị hay không
290         {
291             double xValue = DateTimeAxis.ToDouble(feed.Time); // Chuyển đổi thời gian thành giá trị số
292             tempSeries.Points.Add(new DataPoint(xValue, feed.Temperature.Value)); // Thêm điểm dữ liệu vào biểu đồ
293         }
294     }
295
296     model.Series.Add(tempSeries);
297     pvTemperature.Model = model;
298 }
299

```

Hàm `DateAxis_AxisChanged()`: điều chỉnh định dạng hiển thị thời gian trên trục thời gian sao cho phù hợp với phạm vi thời gian hiện tại (năm, tháng ngày, giờ) trên bản đồ

```

234 // Hàm xử lý sự kiện thay đổi trục thời gian
235 2 references
236 private void DateAxis_AxisChanged(object sender, AxisChangedEventArgs e)
237 {
238     var axis = sender as DateTimeAxis;
239     if (axis == null) return;
240
241     double range = axis.ActualMaximum - axis.ActualMinimum; // Tính khoảng cách giữa giá trị tối đa và tối thiểu
242
243     if (range > 365) // > 1 năm
244         axis.StringFormat = "yyyy";
245     else if (range > 31) // > 1 tháng
246         axis.StringFormat = "MM/yyyy";
247     else if (range > 1) // > 1 ngày
248         axis.StringFormat = "dd/MM";
249     else if (range > 1.0 / 24) // > 1 giờ
250         axis.StringFormat = "HH:mm";
251     else // < 1 giờ
252         axis.StringFormat = "HH:mm:ss";
253 }
254

```

Hàm PlotHumidity(): Vẽ biểu đồ độ ẩm hoạt động tương tự như hàm PlotTemperature()

```
299
300 // Hàm vẽ biểu đồ độ ẩm
301 1 reference
302 private void PlotHumidity(List<DateTime Time, float? Temperature, float? Humidity> feeds)
303 {
304     var model = new PlotModel { Title = "Biểu đồ Độ ẩm (Giờ Việt Nam) " };
305     var dateAxis = new DateTimeAxis
306     {
307         Position = AxisPosition.Bottom,
308         Title = "Thời gian",
309         IntervalType = DateTimeIntervalType.Auto,
310         MajorGridLineStyle = LineStyle.Solid,
311         MinorGridLineStyle = LineStyle.Dot
312     };
313     dateAxis.AxisChanged += DateAxis_AxisChanged;
314     model.Axes.Add(dateAxis);
315     model.Axes.Add(new LinearAxis
316     {
317         Position = AxisPosition.Left,
318         Title = "Độ ẩm (%)",
319         Minimum = 0,
320         Maximum = 50
321     });
322     var humSeries = new LineSeries
323     {
324         Title = "Độ ẩm",
325         Color = OxyColors.DeepSkyBlue
326     };
327     foreach (var feed in feeds)
328     {
329         if (feed.Humidity.HasValue)
330         {
331             double xValue = DateAxis.ToDouble(feed.Time);
332             humSeries.Points.Add(new DataPoint(xValue, feed.Humidity.Value));
333         }
334     }
335     model.Series.Add(humSeries);
336     pvHumidity.Model = model;
337 }
338
339
340
341
342 }
```

UserInfo.cs: hiển thị thông tin người dùng và thiết bị, đồng thời cung cấp phương thức cập nhật và thay đổi thông tin.

Hàm InitializeUser(): sử dụng để lấy các trường thông tin người dùng từ cơ sở dữ liệu

Thực hiện mở kết nối tới cơ sở dữ liệu supabase bằng

Sử dụng lệnh truy vấn “*SELECT * FROM users WHERE email = @email*” để lấy tất cả thông tin của người dùng trên cơ sở dữ liệu thông qua giá trị email của họ

Ghi các giá trị nhận được cho textbox tương ứng và hiển thị lên giao diện

```

34 3 references
35 private void InitializeUser()
36 {
37     lblCheckName.Text = lblCheckSdt.Text = lblCheckEmail.Text = lblCheckConfirm.Text = lblStatus.Text = "";
38     lblcheckPass.ForeColor = Color.FromArgb(102, 102, 102);
39
40     try
41     {
42         using (var conn = new NpgsqlConnection(connString))
43         {
44             conn.Open();
45             string query = "SELECT * FROM users WHERE email = @email";
46             using (var cmd = new NpgsqlCommand(query, conn))
47             {
48                 cmd.Parameters.AddWithValue("@email", srcEmail);
49                 using (var reader = cmd.ExecuteReader())
50                 {
51                     if (reader.Read())
52                     {
53                         idUser = reader["id_user"].ToString();
54                         tbNameUser.Text = lblNameUser.Text = reader["name_user"].ToString();
55                         tbSdt.Text = lblSdt.Text = reader["phone_number"].ToString();
56                         tbEmail.Text = lblEmail.Text = reader["email"].ToString();
57                         tbEmail.Enabled = false;
58                         tbPass.Text = reader["password"].ToString();
59                         tbConfirm.Text = reader["password"].ToString();
60                     }
61                 }
62             }
63         }
64     }
65 }

```

Hàm InitializeDevice(): sử dụng để lấy các thông tin thiết bị tương ứng của người dùng tương tự như hàm **InitializeUser()** bằng cách thực hiện lệnh truy vấn in ra các trường thông tin của thiết bị dựa vào email của họ, sau đó hiển thị giá trị đó lên giao diện

```

3 references
70 private void InitializeDevice()
71 {
72     lblCheckNameDevice.Text = lblCheckIDChannel.Text = lblCheckAPIKey.Text = "";
73
74     try
75     {
76         using (var conn = new NpgsqlConnection(connString))
77         {
78             conn.Open();
79             string query = "SELECT * FROM devices WHERE id_user = @id_user";
80             using (var cmd = new NpgsqlCommand(query, conn))
81             {
82                 cmd.Parameters.AddWithValue("@id_user", int.Parse(idUser));
83                 using (var reader = cmd.ExecuteReader())
84                 {
85                     if (reader.Read())
86                     {
87                         tbNameDevice.Text = reader["device_name"].ToString();
88                         tbAPIKey.Text = reader["api_key"].ToString();
89                         tbIDChannel.Text = reader["channel_id"].ToString();
90                         tbDescription.Text = reader["description"].ToString();
91                     }
92                     else
93                     {
94                         tbNameDevice.Text = "";
95                         tbAPIKey.Text = "";
96                         tbIDChannel.Text = "";
97                         tbDescription.Text = "";
98                     }
99                 }
100             }
101         }
102     }
103     catch (Exception ex)
104     {
105         MessageBox.Show("Lỗi khi tải thông tin thiết bị: " + ex.Message);
106     }
107 }

```

Hàm DeviceExists(): được dùng để kiểm tra xem thiết bị trong cơ sở liệu chưa, bằng cách đếm số lượng thiết bị của người dùng thông qua email của họ, nếu giá trị lớn hơn 0 chứng tỏ rằng họ đã có ít nhất 1 thiết bị trong dữ liệu.

```

108
109 // Hàm kiểm tra thiết bị đã tồn tại trong cơ sở dữ liệu chưa
110 private bool DeviceExists()
111 {
112     try
113     {
114         using (var conn = new NpgsqlConnection(connString))
115         {
116             conn.Open();
117             string query = "SELECT COUNT(*) FROM devices WHERE id_user = @id_user";
118             using (var cmd = new NpgsqlCommand(query, conn))
119             {
120                 cmd.Parameters.AddWithValue("@id_user", int.Parse(idUser));
121                 long count = (long)cmd.ExecuteScalar();
122                 return count > 0;
123             }
124         }
125     }
126     catch (NpgsqlException ex)
127     {
128         MessageBox.Show($"Lỗi kiểm tra thiết bị: {ex.Message}");
129         return false;
130     }
131 }

```

Hàm btnSave_Click(): dùng để cập nhật thông tin thiết bị và người dùng

Kiểm tra xem các trường thông tin người dùng và thiết bị được nhập đã chính xác chưa.

Thực hiện mở kết nối đến supabase và sử dụng lệnh truy vấn “*UPDATE users SET name_user = @name_user, phone_number = @phone_number, password = @password WHERE id_user = @id_user*” để cập nhật dữ liệu người dùng tương ứng với giá trị họ điền trong textbox.

Sau đó sử dụng phương thức **ExecuteNonQuery()** để xác định xem đã cập nhật thuộc tính người dùng thành công chưa, nếu kết quả là 0 nghĩa là không có hàng nào thay đổi chứng tỏ rằng việc cập nhật thông tin thất bại.

Đối với thiết bị, ta cần kiểm tra người dùng đã có thiết bị trong dữ liệu chưa trước khi cập nhật để tránh lỗi.

Nếu chưa ta thực hiện “*INSERT INTO devices (id_user, device_name, channel_id, api_key, description) VALUES (@id_user, @device_name, @channel_id, @api_key, @description)*” để thêm thông tin thiết bị của người dùng vào cơ sở dữ liệu

Nếu rồi, thực hiện truy vấn *UPDATE* để cập nhật các giá trị thuộc tính cho thiết bị nhập từ người dùng rồi gọi hàm **InitializeUser()** và **InitializeDevice()** để cập nhật giao diện.


```

1 reference
227 private void btnSave_Click(object sender, EventArgs e)
228 {
229     if (!checkUserInfo() || !checkDevice())
230     {
231         return;
232     }
233
234     try
235     {
236         using (var conn = new NpgsqlConnection(connString))
237         {
238             conn.Open();
239
240             // Cập nhật thông tin người dùng trong bảng users
241             string userQuery = "UPDATE users SET name_user = @name_user, phone_number = @phone_number,
242             using (var userCmd = new NpgsqlCommand(userQuery, conn))
243             {
244                 userCmd.Parameters.AddWithValue("@id_user", int.Parse(idUser));
245                 userCmd.Parameters.AddWithValue("@name_user", tbNameUser.Text.Trim());
246                 userCmd.Parameters.AddWithValue("@phone_number", tbSdt.Text.Trim());
247                 userCmd.Parameters.AddWithValue("@password", tbPass.Text.Trim());
248
249                 int userRowsAffected = userCmd.ExecuteNonQuery();
250                 if (userRowsAffected == 0)
251                 {
252                     lblStatus.Text = "Không thể cập nhật thông tin người dùng!";
253                     lblStatus.ForeColor = Color.Red;
254                     return;
255                 }
256             }

```

```

257
258     if (DeviceExists())
259     {
260         // Nếu thiết bị đã tồn tại, thực hiện UPDATE
261         string deviceQuery = "UPDATE devices SET device_name = @device_name, channel_id = @channel_id, api_key
262         using (var deviceCmd = new NpgsqlCommand(deviceQuery, conn))
263         {
264             deviceCmd.Parameters.AddWithValue("@id_user", int.Parse(idUser));
265             deviceCmd.Parameters.AddWithValue("@device_name", tbNameDevice.Text.Trim());
266             deviceCmd.Parameters.AddWithValue("@channel_id", tbIDChannel.Text.Trim());
267             deviceCmd.Parameters.AddWithValue("@api_key", tbAPIKey.Text.Trim());
268             deviceCmd.Parameters.AddWithValue("@description", tbDescription.Text.Trim());
269
270             int deviceRowsAffected = deviceCmd.ExecuteNonQuery();
271             if (deviceRowsAffected == 0)
272             {
273                 lblStatus.Text = "Không thể cập nhật thông tin thiết bị!";
274                 lblStatus.ForeColor = Color.Red;
275                 return;
276             }
277         }
278     }
279     else
280     {
281         // Nếu thiết bị chưa tồn tại, thực hiện INSERT
282         string insertDeviceQuery = "INSERT INTO devices (id_user, device_name, channel_id, api_key, description
283         using (var deviceCmd = new NpgsqlCommand(insertDeviceQuery, conn))
284         {
285             deviceCmd.Parameters.AddWithValue("@id_user", int.Parse(idUser));
286             deviceCmd.Parameters.AddWithValue("@device_name", tbNameDevice.Text.Trim());
287             deviceCmd.Parameters.AddWithValue("@channel_id", tbIDChannel.Text.Trim());
288             deviceCmd.Parameters.AddWithValue("@api_key", tbAPIKey.Text.Trim());
289             deviceCmd.Parameters.AddWithValue("@description", tbDescription.Text.Trim());
290
291             deviceCmd.ExecuteNonQuery();
292         }
293     }
294
295     InitializeUser();
296     InitializeDevice();
297
298     lblStatus.Text = "Cập nhật thông tin thành công!";
299 }
300

```

Chatbot.cs: thực hiện chức năng tạo một kênh chatbot đánh giá và thảo luận trạng thái nhiệt độ và độ ẩm với người dùng

Khai báo biến toàn cục

Biến cố định

- httpClient: đối tượng để gửi yêu cầu HTTP đến Gemini API.
- geminiApiKey: khóa API dùng để xác thực với dịch vụ của Gemini

Biến trạng thái

- currentTemperature và currentHumidity: lưu trữ giá trị nhiệt độ và độ ẩm hiện tại.
- prevTemperature và prevHumidity: lưu trữ giá trị trước đó để so sánh sự thay đổi và cập nhật lại giá trị mới.
- isRoleDefined: cờ theo dõi xem vai trò của chatbot đã được định nghĩa với Gemini chưa.

```
13     private readonly HttpClient httpClient = new HttpClient();
14     private readonly string geminiApiKey = "AIzaSyDYgaiUtUoh4HYayXW6UqYIVj34Lsqdshk"; // API key cho Gemini API
15
16     // Biến lưu trữ dữ liệu môi trường từ form MenuForm
17     private string currentTemperature = "Không có dữ liệu";
18     private string currentHumidity = "Không có dữ liệu";
19
20     // Biến lưu trữ giá trị trước đó của nhiệt độ và độ ẩm để kiểm tra sự thay đổi
21     private string prevTemperature = null;
22     private string prevHumidity = null;
23
24     private bool isRoleDefined = false; // Cờ để kiểm tra xem vai trò đã được định nghĩa chưa
25
```

Hàm UpdateTemperatureAndHumidity(): nhận dữ liệu nhiệt độ và độ ẩm từ MenuForm (Form lấy dữ liệu trên ThingSpeak), nếu dữ liệu đó khác biệt với dữ liệu nhận được trước đó thì sẽ được cập nhật lại.

```

31
32 // Cập nhật dữ liệu môi trường từ form MenuForm
33 public void UpdateTemperatureAndHumidity(string temperature, string humidity)
34 {
35     currentTemperature = temperature;
36     currentHumidity = humidity;
37
38     // Chỉ cập nhật dữ liệu nếu dữ liệu thay đổi
39     if (currentTemperature != prevTemperature || currentHumidity != prevHumidity)
40     {
41         prevTemperature = currentTemperature;
42         prevHumidity = currentHumidity;
43     }
44 }

```

Hàm btnSend_Click(): gửi câu hỏi và nhận phản hồi từ Chatbot

Lấy nội dung từ textbox tbChat sau đó tạo prompt cho Chatbot

- Nếu isRoleDefined là false (lần đầu tiên), thêm vai trò (DefineRole()) và dữ liệu môi trường (GetCurrentEnvironmentData()).
- Sau lần đầu, chỉ thêm dữ liệu môi trường và câu hỏi.

Gọi GetGeminiResponse để lấy phản hồi từ Gemini (sử dụng await để xử lý yêu cầu API không chặn luồng giao diện), và hiển thị nội dung trên rich textbox rtbMessages.

```

46 // Phương thức gửi tin nhắn đến ChatBot
47 private async void btnSend_Click(object sender, EventArgs e)
48 {
49     string userInput = tbChat.Text.Trim();
50     if (string.IsNullOrEmpty(userInput))
51         return;
52
53     AppendMessage($"Bạn: {userInput}\n"); // Hiển thị câu hỏi của người dùng trên RichTextBox
54     string prompt = !isRoleDefined ? DefineRole() + GetCurrentEnvironmentData() + $"Câu hỏi: {userInput}" // Định nghĩa vai trò và gửi câu hỏi
55         : GetCurrentEnvironmentData() + $"Câu hỏi: {userInput}";
56     tbChat.Text = "";
57     isRoleDefined = true;
58
59     string response = await GetGeminiResponse(prompt); // Nhận phản hồi từ Gemini
60     AppendMessage($"ChatBot: {response}\n\n"); // Hiển thị phản hồi từ Gemini trên RichTextBox
61 }

```

Hàm btnRefresh_Click(): bắt đầu lại trò chuyện mới bằng cách xóa nội dung trò chuyện hiện tại ở rtbMessages và đặt lại isRoleDefined = false để định nghĩa lại vai trò cho Chatbot trong lần gửi đầu tiên

```

63 // Phương thức tạo cuộc trò chuyện mới
64 private void btnRefresh_Click(object sender, EventArgs e)
65 {
66     rtbMessages.Clear();
67     isRoleDefined = false;
68
69     StartNewConversation(); // Bắt đầu cuộc trò chuyện mới
70 }

```

Hàm StartNewConversation(): thông báo chào hỏi trước khi bắt đầu cuộc trò chuyện

Hàm DefineRole(): định nghĩa vai trò cho Chatbot

Hàm GetCurrentEnvironmentData(): cung cấp dữ liệu môi trường hiện tại (nhiệt độ và độ ẩm) cho Chatbot

```
80 // Lỗi giới thiệu khi bắt đầu cuộc trò chuyện mới
81
82 2 references
83 private void StartNewConversation()
84 {
85     AppendMessage($"ChatBot: Xin chào! Tôi là ChatBot, sẵn sàng giúp bạn đánh giá trạng thái môi trường hiện tại.\n");
86 }
87
88 // Định nghĩa vai trò cho Gemini
89 1 reference
90 private string DefineRole()
91 {
92     return "Bạn là một trợ lý AI đánh giá trạng thái môi trường dựa trên dữ liệu nhiệt độ và độ ẩm. Cung cấp phản tích ngắn gọn (tối đa 150 từ) và khuyến nghị nếu cần. ";
93 }
94
95 // Lấy dữ liệu môi trường hiện tại
96 2 references
97 private string GetCurrentEnvironmentData()
98 {
99     return $"Dữ liệu môi trường hiện tại: Nhiệt độ = {prevTemperature}°C, Độ ẩm = {prevHumidity}%.";
100 }
```

Hàm GetGeminiResponse(): gửi yêu cầu đến Gemini API và nhận phản hồi

Tạo URL: sử dụng endpoint generateContent của mô hình gemini-1.5-flash với geminiApiKey.

Tạo request body:

- Cấu trúc JSON với contents chứa parts, trong đó text là prompt.
- Sử dụng Newtonsoft.Json.JsonConvert.SerializeObject để chuyển thành chuỗi JSON để gửi yêu cầu đến API

Gửi yêu cầu:

- Sử dụng httpClient.PostAsync để gửi yêu cầu POST bất đồng bộ.
- EnsureSuccessStatusCode(): kiểm tra mã trạng thái HTTP (ném ngoại lệ nếu không thành công).

Xử lý phản hồi:

- Đọc nội dung phản hồi từ API dưới dạng chuỗi JSON từ response.Content.
- Chuyển đổi chuỗi JSON thành đối tượng JObject để dễ dàng truy cập các thuộc tính bằng JObject.Parse
- Trích xuất nội dung phản hồi từ candidates[0].content.parts[0].text.

- Nếu không có dữ liệu, trả về thông báo "Không nhận được phản hồi từ Gemini."

```

99 // GỬI yêu cầu đến Gemini API và nhận phản hồi
100 1 reference
101 private async Task<string> GetGeminiResponse(string prompt)
102 {
103     try
104     {
105         string url = $"https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key={geminiApiKey}"; // URL của Gemini API với tham số là
106         // Tạo đối tượng với cấu trúc JSON theo định dạng yêu cầu của Gemini API
107         var requestBody = new
108         {
109             contents = new[]
110             {
111                 new
112                 {
113                     parts = new[]
114                     {
115                         new { text = prompt }
116                     }
117                 }
118             }
119         };
120
121         var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(requestBody), System.Text.Encoding.UTF8, "application/json"); // Chuyển đổi đối tượng
122         var response = await httpClient.PostAsync(url, content); // Gửi yêu cầu POST bất đồng bộ đến URL đã chỉ định, với nội dung là content
123         response.EnsureSuccessStatusCode(); // Kiểm tra xem phản hồi có mã trạng thái HTTP thành công không
124
125         string responseBody = await response.Content.ReadAsStringAsync(); // Đọc nội dung phản hồi từ API dưới dạng chuỗi JSON
126         JObject jsonResponse = JObject.Parse(responseBody); // Chuyển đổi chuỗi JSON thành đối tượng JObject để dễ dàng truy cập các thuộc tính
127
128         // Trích xuất nội dung phản hồi từ Gemini
129         string result = jsonResponse["candidates"]?[0]?["content"]?[0]?["parts"]?[0]?["text"]?.ToString();
130         return result ?? "Không nhận được phản hồi từ Gemini.";
131     }
132     catch (Exception ex)
133     {
134         return $"Lỗi: {ex.Message}";
135     }
136 }

```

Hàm AppendMessage(): thêm tin nhắn của người dùng hoặc chatbot vào richtextbox để hiển thị các tin nhắn hiện tại và trước đó

```

137
138 // Phương thức thêm tin nhắn vào RichTextBox
139 3 references
140 private void AppendMessage(string message)
141 {
142     rtbMessages.SelectionStart = rtbMessages.TextLength;
143     rtbMessages.SelectionLength = 0;
144
145     rtbMessages.AppendText(message);
146     rtbMessages.ScrollToCaret();
147 }
148 }

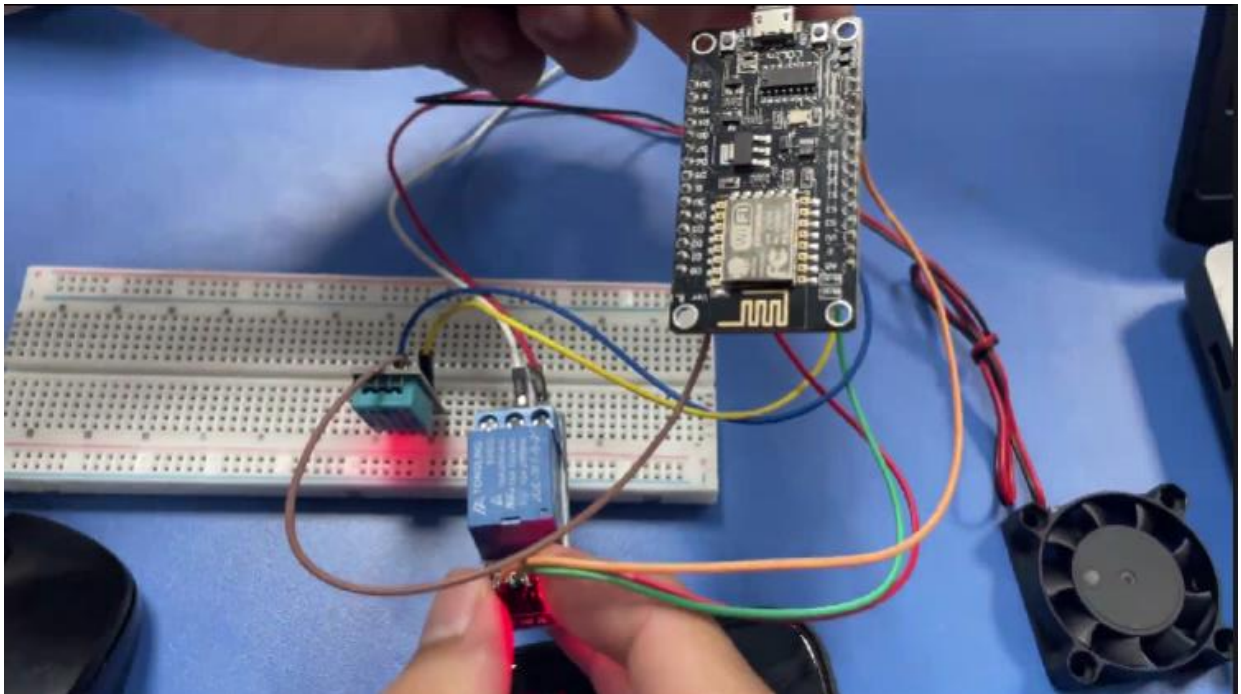
```

Chương 3. TRIỂN KHAI VÀ DEMO

3.1. Hình ảnh minh họa

Lắp ráp bảng mạch và nạp chương trình

Ta thực hiện lắp ráp bảng mạch theo sơ đồ, rồi kết nối với máy tính để nạp chương trình vào module ESP8266.



Nhận dữ liệu thông qua API của ThingSpeak

Sau khi cảm biến đã thu thập dữ liệu hoàn tất, ESP8266 sẽ gửi dữ liệu lên ThingSpeak đến tài khoản của người dùng, thông qua API ta có thể xem được các trường thông tin như giá trị nhiệt độ và độ ẩm của mỗi bản ghi mà module gửi đến.

```
← ↻ 🏠 🔒 https://api.thingspeak.com/channels/2943515/feeds.json?api_key=LRSNX2AOSRS49MZQ&results=1 ☆ ⚙️ 📄 🕒
Pretty-print
{"channel":{"id":2943515,"name":"Temperature and Humidity Monitoring","latitude":"0.0","longitude":"0.0","field1":"Temperature","field2":"Humidity","field3":"Relay","created_at":"2025-05-01T05:42:32Z","updated_at":"2025-05-09T14:38:24Z","last_entry_id":1583},"feeds":[{"created_at":"2025-05-20T09:44:42Z","entry_id":1583,"field1":"27.60000","field2":"57.00000","field3":"0"}]}
```

Bây giờ vào phần mềm nhóm tự thiết kế để tạo tài khoản, đăng nhập, điền các thông tin người dùng và thiết bị, để xem kết quả.

Đăng kí

Ta thực hiện tạo tài khoản với địa chỉ email cá nhân của mình (tnvhoang2005@gmail.com) sau đó bấm Enter hoặc mũi tên để nhận mã OTP từ email.

×



Đăng kí

Bạn đã có tài khoản ? [Đăng nhập](#)

Địa chỉ email

→

Mã OTP

↺

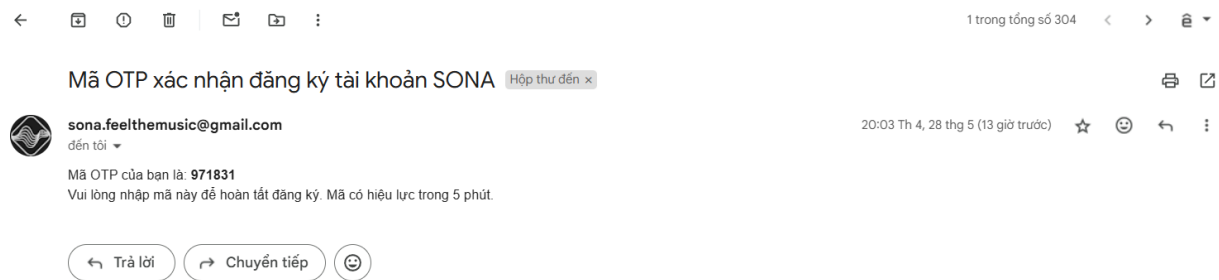
Mã OTP đã được gửi tới Email của bạn!

Đăng kí

Hoặc

 Đăng kí bằng Google

Đăng nhập vào tài khoản gmail ta đã nhập trước đó, ở đây đã xuất hiện tin nhắn gửi OTP đến, ta sử dụng mã đó để tiếp tục thao tác đăng kí.



Ta tiếp tục điền thông tin bao gồm tên người dùng, số điện thoại, mật khẩu ... để hoàn tất việc đăng kí.

×

Bạn đã có tài khoản? Đăng nhập

Chọn ảnh đại diện

Tên người dùng:

Trần Nguyễn Việt Hoàng

Số điện thoại

0123456789

Mật khẩu:

.....

Xác nhận mật khẩu:

.....

Mật khẩu của bạn phải có ít nhất

- 1 chữ cái

- 1 chữ số hoặc ký tự đặc biệt (ví dụ @, !, ?, #)

- 8 ký tự

Đăng kí

Khi này các trường thông tin từ người dùng nhập sẽ được chuyển tới cơ sở dữ liệu Supabase, thuộc tính `id_user` sẽ tự động đánh dấu id của người dùng và `created_at` sẽ ghi lại thời gian người dùng đăng kí.

Filter		Sort		Insert		RLS disabled		Role postgres		Realtime off	
	id	name_user	email	phone_number	password	created_at					
	1	Trần Nguyễn Việt Hoàng	tnvhoang2005@gmail.com	0123456789	Hoang2k5@	2025-05-28 13:05:52.427295					

Đăng nhập

Bây giờ ta có thể sử dụng tài khoản vừa đăng kí để thực hiện thao tác đăng nhập, ở đây ta cũng có thể sử dụng phương thức đăng nhập bằng Google dựa trên tài khoản email đăng kí trước đó.

Đăng nhập

 Đăng nhập bằng Google

Hoặc

Địa chỉ email hoặc tên người dùng

tnvhoang2005@gmail.com

Mật khẩu

 Hiện

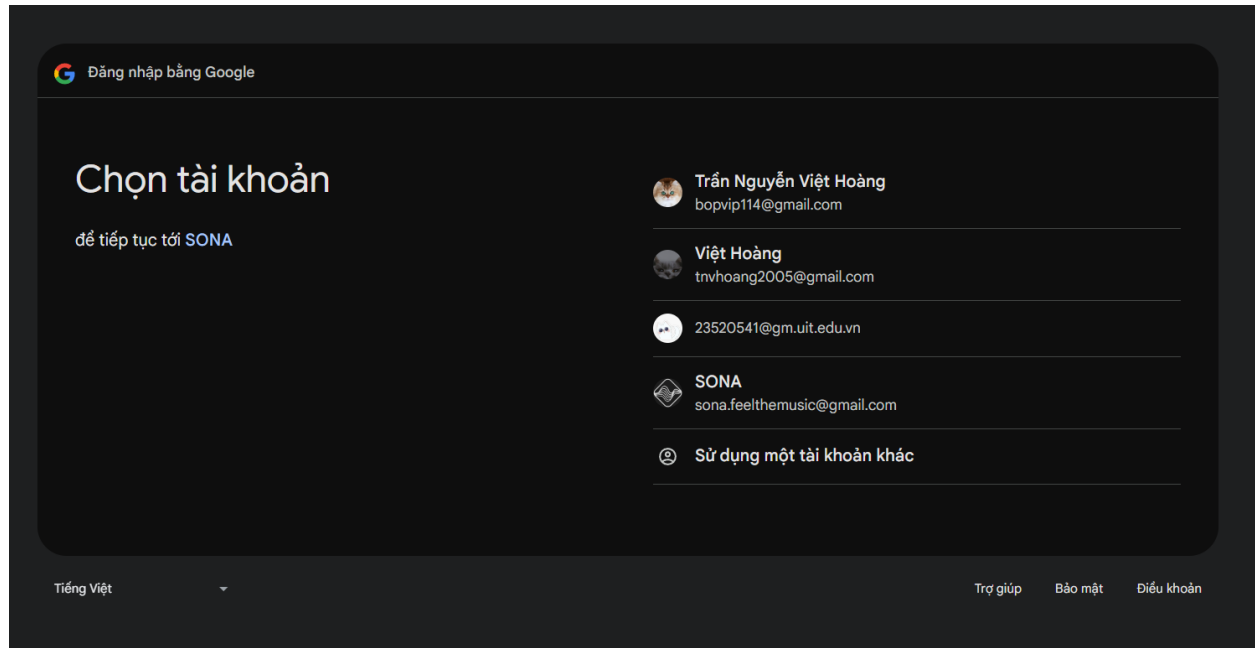
.....

[Quên mật khẩu?](#)

Đăng nhập

Bạn chưa có tài khoản? [Đăng kí](#)

Khi này, giao diện “Đăng nhập bằng Google” sẽ được hiển thị để người dùng chọn tài khoản Google mà ta muốn đăng nhập, do ta đăng kí với tài khoản là tnvhoang2005@gmail.com nên chỉ có tài khoản đó mới được phép sử dụng để đăng nhập



Giao diện chính

Sau khi Login hoàn tất, ta cần phải cập nhật thông tin thiết bị của mình bao gồm: tên thiết bị, id channel và api key

Device management

Tên thiết bị:

ESP8266

ID channel:

2943515

API key:

LRSNX2AOSRS49MZQ

Mô tả:

Thiết bị đo nhiệt độ và độ ẩm

Lưu

Đặt lại

Thoát

Khi này trên cơ sở dữ liệu supabase cũng sẽ cập nhật được thiết bị của người dùng.

id...	api_key	channel_id	id...	device_name	description	created_at
1	LRSNX2AOSRS49MZQ	2943515	1	ESP8266	Thiết bị đo nhiệt độ và độ ẩm	2025-05-28 13:09:09.197606

Giao diện Home hiển thị trạng thái của thiết bị, thời gian nhận dữ liệu, trạng thái relay và các giá trị nhiệt độ và độ ẩm

Giám sát Thiết bị đo nhiệt độ và độ ẩm Real-time

Trạng thái ESP8266: Offline

Dữ liệu được làm mới lúc: 2025-05-20 04:44:42 PM



Trạng thái Relay

Tắt

Nhiệt độ

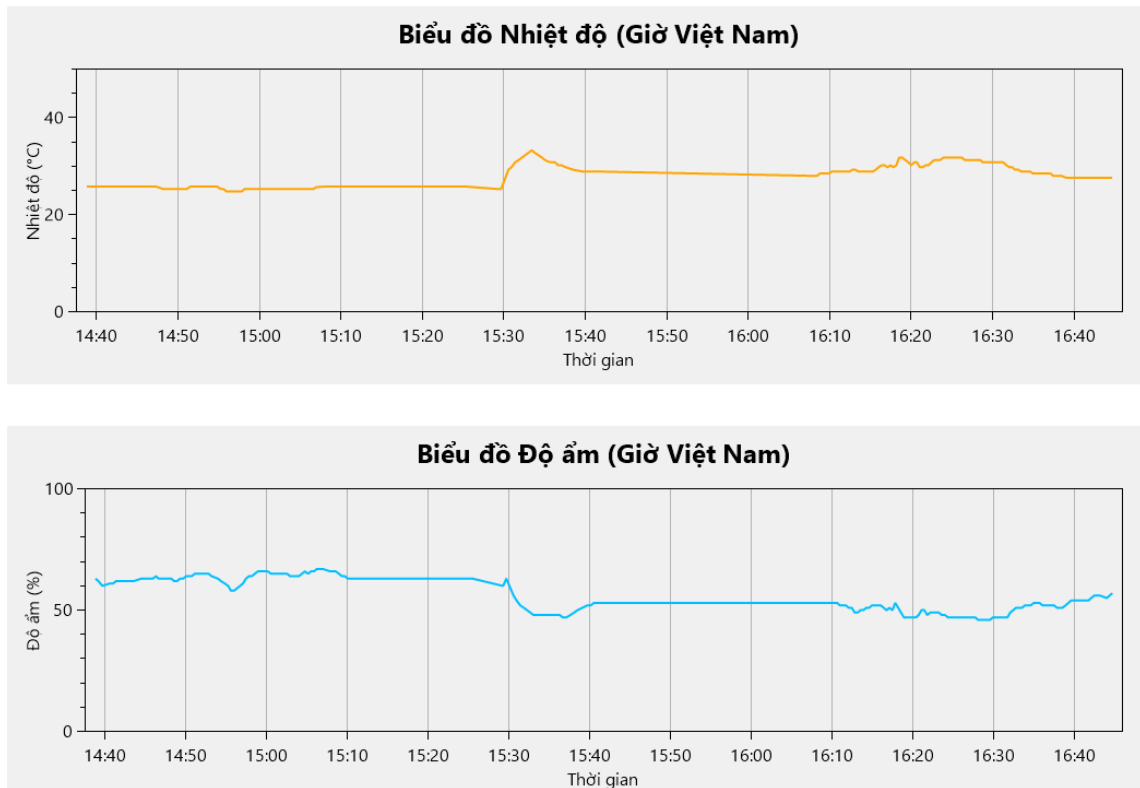
27.6°C

Độ ẩm

57%

Biểu đồ biểu diễn nhiệt độ và độ ẩm trong 2 giờ trước đó

Biểu đồ



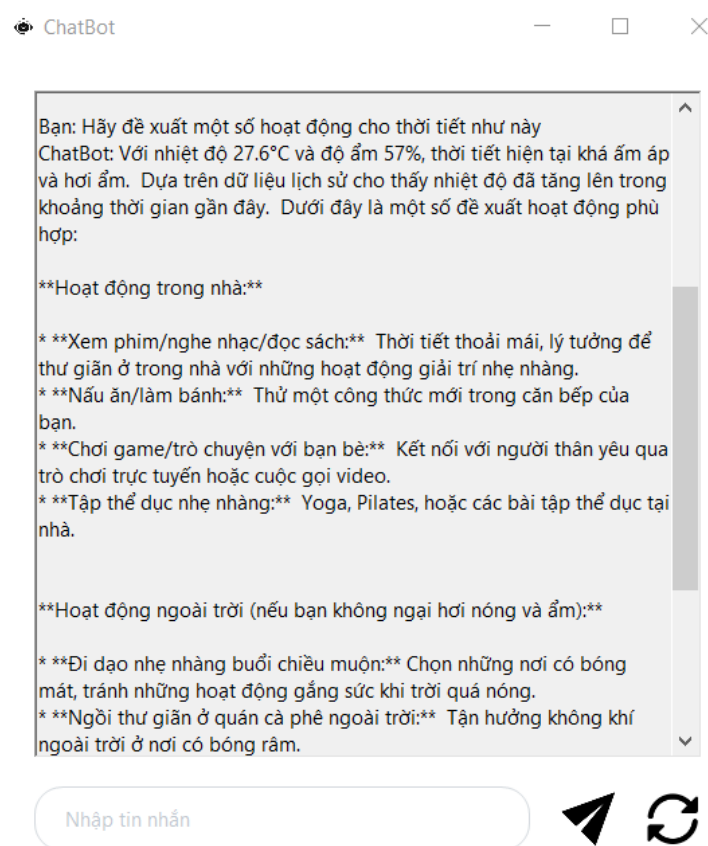
Gửi cảnh báo khi vượt ngưỡng

Kiểm thử tính năng gửi email cảnh báo đến tài khoản gmail người dùng khi nhiệt độ vượt ngưỡng



Trò chuyện với ChatBot

Dựa vào dữ liệu nhiệt độ và độ ẩm mà ChatBot nhận được, ta có thể đặt một số câu hỏi như phân tích dữ liệu hay dự đoán xu hướng của môi trường, cũng có thể đơn giản hơn là một số khuyến nghị với thời tiết hiện tại



3.2. Video demo

Link:

https://drive.google.com/file/d/1MyHDS4Q1o2l0Dkak55blB6GRVfGr3TY/view?usp=s_haring

Chương 4. KẾT LUẬN

4.1. Nhận xét về đồ án

Thành tựu đạt được

Thứ nhất, nhóm đã thành công trong việc thiết kế và triển khai một hệ thống IoT hoàn chỉnh, xây dựng bản mạch với module ESP8266 kết hợp cảm biến DHT11 và Relay 5V để thu thập dữ liệu nhiệt độ và độ ẩm, kết nối mạng Wi-Fi.

Thứ hai, nhóm đã thành công trong việc thiết kế một chương trình trên Arduino IDE để nạp vào module. Chương trình thực hiện chức năng nhận và gửi dữ liệu trên ThingSpeak, đồng thời xây dựng hàm điều khiển quạt tự động khi nhiệt độ vượt ngưỡng.

Thứ ba, nhóm đã xây dựng được một cơ sở dữ liệu Server dùng để quản lý thông tin và thiết bị người dùng trên Supabase.

Thứ bốn, nhóm đã xây dựng được một giao diện người dùng bằng .NET framework có chức năng thu thập dữ liệu từ ThingSpeak và thành công hiển thị trạng thái của mô đun, thông tin nhiệt độ và độ ẩm, đồng thời tích hợp chatbot đánh giá môi trường hiện tại và trò chuyện với người dùng.

Hạn chế của đồ án

Cảm biến DHT11 có độ chính xác không cao (sai số $\pm 2^{\circ}\text{C}$ cho nhiệt độ và $\pm 5\%$ RH cho độ ẩm) và phạm vi đo hạn chế ($0-50^{\circ}\text{C}$, $20-90\%$ RH), điều này có thể ảnh hưởng đến tính chính xác trong các môi trường khắc nghiệt hoặc yêu cầu độ chính xác cao. Ngoài ra, tần

số lấy mẫu của DHT11 chỉ đạt tối đa 1Hz (1 lần/giây), dẫn đến việc hệ thống không thể phản ứng tức thời với những thay đổi nhanh của môi trường.

Hệ thống hiện tại phụ thuộc dữ liệu từ ThingSpeak, buộc người dùng phải có tài khoản ThingSpeak, và người dùng phải gửi dữ liệu từ ESP8266 đến ThingSpeak thay vì gửi trực tiếp cho hệ thống của nhóm. Sau đó người dùng phải cung cấp API và channel ID từ ThingSpeak để nhóm mới có thể lấy được dữ liệu và xử lý, việc này sẽ gây ra sự bất tiện không cần thiết cho người dùng.

4.2. Hướng phát triển sau này

Để cải thiện hệ thống, nhóm đề xuất một số định hướng phát triển trong tương lai. Đầu tiên, có thể thay thế cảm biến DHT11 bằng các cảm biến có độ chính xác cao hơn như DHT22 hoặc BME280, đồng thời mở rộng phạm vi đo để phù hợp với nhiều điều kiện môi trường khác nhau.

Thứ hai, hệ thống cần thiết lập một Socket Client – Server giúp nhận dữ liệu trực tiếp từ module ESP8266 thay vì phụ thuộc vào ThingSpeak, điều này sẽ giúp người dùng loại bỏ các bước rườm rà (như lấy ID channel và API key từ ThingSpeak) mà chỉ cần sử dụng hệ thống nhóm xây dựng là có thể theo dõi các dữ liệu mà thiết bị đo được một cách nhanh chóng và tiện lợi.

Thứ ba, hệ thống của nhóm mới chỉ triển khai trên ứng dụng cho hệ điều hành Windows, nhóm sẽ triển khai hệ thống này lên Website và ứng dụng di động trong thời gian sau này.

Cuối cùng, hệ thống sẽ tích hợp thêm các tính năng AI nâng cao, như trợ lý ảo có thể giao tiếp người dùng bằng giọng nói, hay tự động hóa quá trình tổng hợp và phân tích dữ liệu để dự đoán xu hướng môi trường trong khoảng thời gian tới.

4.3. Đường dẫn chứa các file trong báo cáo

Google Drive (file báo cáo, video demo, source code Arduino):

<https://drive.google.com/drive/folders/1lCU1ejGaCpXqA4DWpoKODXGc1CIXr-jC?usp=sharing>

Canva (slide thuyết trình):

https://www.canva.com/design/DAGkgd91lhg/TEe2NvPdWxJqlCRbQcBxXQ/edit?utm_content=DAGkgd91lhg&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Github: (source code giao diện người dùng)

<https://github.com/Viet-Hoang-2005/ESP8266>

TÀI LIỆU THAM KHẢO

- [1] Jetking, "ESP8266 là gì? Tìm hiểu về module Wi-Fi phổ biến cho IoT," 25 10 2024. [Online]. Available: <https://jetking.fpt.edu.vn/esp8266/>. [Accessed 14 5 2025].
- [2] V. FE, "Cảm Biến Nhiệt Độ Và Độ Ẩm DHT11," 24 10 2024. [Online]. Available: <https://dientutuonglai.com/cam-bien-nhiet-do-va-do-am-dht11.html>. [Accessed 12 5 2025].
- [3] ELProCus, "What is a 5V Relay Module : Working & Its Applications," 30 12 2024. [Online]. Available: <https://www.elprocus.com/5v-relay-module/>. [Accessed 13 5 2025].
- [4] thegioididong, "Arduino IDE - Phần mềm lập trình Arduino miễn phí đa nền tảng," 7 10 2024. [Online]. Available: <https://www.thegioididong.com/game-app/arduino-ide-phan-mem-lap-trinh-arduino-mien-phi-da-nen-235232>. [Accessed 13 5 2025].
- [5] FPT, "Khám phá về ThingSpeak - Nền tảng IoT cho người mới bắt đầu - ThingSpeak là gì?," 6 5 2024. [Online]. Available: <https://fptshop.com.vn/tin-tuc/danh-gia/thingspeak-181528>. [Accessed 13 5 2025].
- [6] B. Tech, "What is Supabase?," 14 12 2023. [Online]. Available: <https://blog.boldtech.dev/what-is-supabase-vs-firebase/>. [Accessed 14 3 2025].

PHÂN CHIA CÔNG VIỆC

MSSV	Họ và tên	Công việc
23520541	Trần Nguyễn Việt Hoàng	Xây dựng cơ sở dữ liệu Supabase (100%) Xây dựng hệ thống quản lý thiết bị theo người dùng bằng .NET framework (100%) Viết báo cáo (70%)
23521572	Lê Quang Tiến	Lắp ráp bản mạch ESP8266 (50%) Làm slide thuyết trình (100%) Viết báo cáo (30%)
23521778	Lê Hoàng Việt	Lắp ráp bản mạch ESP8266 (50%) Viết chương trình nạp vào module và gửi dữ liệu lên ThingSpeak (100%)